Bean — Architecture Guide

Audience: another bot/engineer who must understand, modify, rebuild and re-publish this extension. Store id: `panel9-bean` *(type* `extension` *, target* `panel-9` *, platform* `any` *). Current version* ***1.1.0*** *(1.0.0 was the first release; it was previously named Comment Liker, never published).* ***Requires Panel 9 2.3.10+*** *(2.3.9 added the container bridge, 2.3.10 the ExpressVPN bridge + background runner); on an older Panel 9 it degrades to single-container behaviour (current tab's container) without VPN rotation.*

1. What it does

The user pastes YouTube comment links in the toolbar popup and ticks the containers (Google accounts) — and, for accounts that own several YouTube channels, the channels — that should like. For each ticked container one tab **in that container** (the same tab, link after link) opens `https://www.youtube.com/watch?v=VID&lc=COMMENT`, finds the linked comment, and presses its Like button **once**, only if it is not already liked. The result per link × container × channel is shown in the popup. It does this **only** if YouTube Focus (`yt-focus` , ≥ 1.2.0 tested) is installed, enabled, and its *Comments only* switch is on.

2. Files

```
manifest.json     MV3. permissions: storage. host_permissions + content-script match:
https://www.youtube.com/* . action.default_popup = options_page = popup.html. CSP script-src 'self'.
lib/links.js      pure helpers shared by popup, content script and node tests: parseLink/parseMany
(watch, youtu.be, shorts, live, embed, replies lc=PARENT.REPLY), watchUrl(vid, lc, batchToken,
containerId, channelId), pageInfo (also the discovery/restore pages), discoverUrl/restoreUrl,
parseSwitcher (YouTube's account-switcher JSON → channels), sameChannel, switchUrl, cacheChannels, chKey,
planActions (container → channel → link), countSwitches, estimateSeconds, caps (MAX_ACTIONS 200,
WARN_ACTIONS 40), pace windows, result codes + labels
content.js        isolated-world content script (document_start, top frame of www.youtube.com): (1)
YouTube Focus detection, (2) the batch runner
popup.html/.css/.js  the UI: container picker as a tree (containers → channels with avatars, tri-state,
per-container and global All/None, Refresh channels, filter, remembered), textarea, pace,
count/estimate/warning line, Like/Stop/Clear, status pill + gate banner, per-link×container results, and
the ORCHESTRATOR that opens one tab per container
theme.css, fonts/    copy of the Matte tokens + Inter (same as the other Panel 9 extensions) so the popup
follows the browser theme
icons/, store-icon-512.png
tests/unit.js     node: link parsing, pacing, static checks (no inline script, one click call, yt-focus
keys still exist)
tests/mock-youtube.js, tests/e2e.js   Electron e2e inside the real Panel 9 with the real yt-focus against
a local mock of youtube.com
```

3. How it knows YouTube Focus' comment-only view is on

Extensions cannot read each other's `chrome.storage` , and Panel 9's privileged `window.panel9ext` bridge does not expose yt-focus' state (it only gives Bean containers/openTab, §5). But **the DOM is shared** between extension content scripts, and yt-focus mirrors its switches onto `<html>` :

marker (set by yt-focus `content.js`)	meaning
`<html data-ytf-comments>`	the **Comments only** switch is ON (what we gate on)
`<html data-ytf-focus>`	…and this page is a watch page rendered in comment-only layout
`<html data-ytf-acct>`	comment-only view found a signed-in avatar (used as "signed in" signal)

DOM event `ytf-state` on `document`, element `#ytf-bar`, any `data-ytf-*` attribute	yt-focus is present/running

`content.js` publishes what it sees to **its own** storage key `p9bn.ytf = {installed, commentsOnly, seen}` (only on change / every 30 s; "cannot tell" is never written — absence of yt-focus is only proven on a `/watch` page after 6 s). The popup writes `p9bn.ping = Date.now()`; every YouTube tab answers immediately by refreshing `p9bn.ytf`, so the popup only trusts an answer whose `seen >= ping`. No fresh answer (no YouTube tab open) = "not checked yet" and the button stays enabled; **the page itself re-checks before every like** (authoritative gate), so a stale popup can never cause a like while the switch is off. If yt-focus renames these markers, `tests/unit.js` fails ("yt-focus detection keys exist…"). Keep them in sync with `/workspace/yt-focus-extension/content.js`. (Possible future improvement needing a Panel 9 change: expose yt-focus' storage state through the p9ext bridge. Not needed now.)

4. The run (popup ⇄ one YouTube tab per container)

Storage keys (all `chrome.storage.local`, mirrored by Panel 9 to every session = container): `p9bn.ytf`, `p9bn.ping`, `p9bn.batch`, `p9bn.settings` (`{pace, sel:{<profileId>:[containerId…]}, chs:{<profileId>:{<containerId>:[channelId…]}}}`), `p9bn.ch.<profileId>.<containerId>` (the channel cache `{at, ok, channels:[{id,name,handle,avatar}], active, err}`), `p9bn.disc` (the validating record of a "Refresh channels" pass) and `p9bn.diag` (a 30-entry log of abnormal events, e.g. a run record that never reached a container).

1. **Targets.** The popup asks the bridge for the containers (`window.panel9ext.containers()` → `{profile, active, containers:[{id,name,color,builtin,signedIn,checked,people}]}`), restores the remembered selection (ids that no longer exist are dropped) and shows the picker. Targets = ticked containers, else the **active** container (the current tab's), else (no bridge) "the container of the active tab" via `window.open`.
2. **Plan.** `planActions(links, targets)` → `items` (container-major: all links of container 1, then container 2…) and `groups` (one per container: `queued → opening → running → done`). links × containers ≤ 200 (`MAX_ACTIONS`; button disabled above), warning above 40. The popup checks the gate (ping) and writes `p9bn.batch = {id, state:'running', pace, created, updated, nextAt, groups[], items:[{n, vid, lc, url, cid, cname, status:'queued'}]}`.
3. **Orchestrator** (`advance()` in popup.js; on storage changes and once a second while the popup is open): when no group is opening/running and `now ≥ batch.nextAt`, it marks the next queued group `opening` and calls `window.panel9ext.openTabInContainer({containerId, url: watchUrl(firstItem, batch.id, containerId)})`. If the bridge refuses/fails the group's items become `open-failed` and the run moves on. A group whose items are all final is `done`, then `nextAt = now + pause(pace)` (the human-like gap between containers). Without the bridge (older Panel 9) there is a single group with `cid:''` opened with `window.open`.
4. **Runner** (content.js, in the tab): the URL carries `&p9bn=<batch id>&p9bc=<container id>`. The tab waits up to 15 s for the run record (the storage mirror reaches each session a moment after the write), then acts only on the queued item whose `vid + lc + cid` match (stale links do nothing). It claims the item (`runner`, 4 s heartbeat `hb`) and runs `processComment()`:
   1. gate: yt-focus present (≤ 6 s) → `data-ytf-comments` set → `data-ytf-focus` set. Else `no-ytf` / `ytf-off` — **checked in every container's tab, before every like**.
   2. signed in? `html[data-ytf-acct]` or `ytd-masthead #avatar-btn` ⇒ in; masthead Sign-in link ⇒ `not-signed-in`.
   3. find the comment: `ytd-comments a[href*="lc="]` whose `lc` equals the id **exactly** → `closest('ytd-comment-view-model, ytd-comment-renderer')`; fallback: the `#linked-comment-badge`. It nudges the section into view (lazy loading); for a reply it opens the parent's `#more-replies`. 35 s timeout ⇒ `not-found`; "turned off" ⇒ `comments-off`.
   4. like button `#like-button button`, state from `aria-pressed`. Read twice 0.8 s apart; `true` ⇒ `already` (no click); unreadable ⇒ `unknown-state` (no click); `false` ⇒ human pause, then **one** `btn.click()`.
   5. verify `aria-pressed` becomes `true` within 5 s ⇒ `liked`; else sign-in dialog ⇒ `not-signed-in`; else `click-failed`. **Never a second click** (a retry could un-like).
5. After each item: record it; `ytf-off` / `no-ytf` ⇒ the **whole run** stops (`state:'stopped'`, every queued item `skipped` with that code — it is the same in every container, no further tab is opened); `not-signed-in` ⇒ only **this container's** remaining items are skipped and the next container still runs. Otherwise a random pause from the pace window (Fast 2–5 s, Normal 4–10 s, Slow 9–20 s) and the **same tab** goes to this container's next link. After the container's last link the group is

`done`, `nextAt` is set, and when nothing is queued/running any more `state:'done'`; the tokens are stripped from the address bar.

6. **Stop** = popup sets `state:'stopped'` and skips queued items; every runner sees it (`storage.onChanged` / 2.5 s poll) and does nothing more; the orchestrator never opens another tab. A running record not touched for 45 s means its tab is gone: the active group's unfinished items become `interrupted` and the orchestrator continues with the next container; if the popup was closed and is reopened **within 5 min** the run resumes, later it is closed as `stopped`. A small shadow-DOM toast on each page shows progress (`Bean Channel A · 2/5 · Liked`).

4b. Channels (several YouTube channels per Google account)

Discovery ("Refresh channels"). For every ticked container (or the current one) the popup writes `p9bn.disc={id,state:'running'}`, opens a short-lived tab in that container at `https://www.youtube.com/?p9bn=<id>&p9bc=<container>&p9bp=<profile>&p9bd=1` (the home page — the bridge allows `https://www.youtube.com/?…` for this), waits for the cache key to be rewritten and closes the tab again through the bridge's `closeTab` (only tabs the bridge opened itself). `content.js` `runDiscover()` validates the id, waits for the page to settle and does **one same-origin `fetch('/getAccountSwitcherEndpoint', {credentials:'same-origin'})` — the call the page itself makes when you open avatar menu → Switch account**. `parseSwitcher()` walks the JSON for `accountItemSectionRenderer → accountItem`, takes the section that contains the `isSelected` item (= the Google account signed in in this container; other Google accounts in the list are ignored), and keeps per channel: id (the `pageId`, or `main`), name, `@handle` only if the byline is a handle (an e-mail byline is dropped), https avatar, active flag and — in memory only — YouTube's own `signinUrl`. `cacheChannels()` stores id/name/handle/avatar only. A failed read (signed out, format changed) keeps the previous good list and sets `err`. No cookie or token is ever read, extracted or stored (a unit test greps for it). **Picker.** `sel` (containers) and `chSel[cid]` (channels) are kept in step: a container is ticked iff ≥ 1 of its channels is ticked (tri-state box); ticking a container ticks all its channels; the first time a list appears only the **active** channel is ticked; channels that disappeared are dropped. Containers without a list are plain checkboxes and like from whatever channel is active there. Single-channel accounts show their channel name and need nothing. **Run order.** `planActions` → container → channel → link, the channel that is active at refresh time first (no switch needed for it). Items carry `chid/chname`, the URL carries `&p9bh=<channelId>`, an item is claimed by `vid+lc+cid+chid`. The cap counts every channel (links × Σ channels ≤ 200, warning > 40); the estimate adds 12 s per switch. **Switching (content.js `runBatch`).** Before each like in a container with a channel list the tab re-reads the switcher: (1) records the container's original channel (`group.orig/origName`, once); (2) if the active channel is the item's channel → like (same single-click path); (3) otherwise it records `tries=1` + a switch counter, clears the item's claim and navigates to YouTube's signin address for that channel (`switchUrl`: the list's own `signinUrl`, else `/signin?action_handle_signin=true&app=desktop&skip_identity_prompt=true`, plus `pageid=<id>` and `next=<watch URL with the run tokens>`); the reloaded tab claims the same item, re-reads the switcher and **verifies** (`sameChannel`: id, then handle, then name). If it is still not the right channel, or the list cannot be read, or the channel is gone → the item ends as `switch-failed` ("Could not switch to channel X") and **no like is attempted** (`processComment` is only reached when no failure result exists; one switch attempt per item, no retry loop). **Restore.** When the last item of a container is done and its channel differs from `orig` (and a switch happened), `group.restore='pending'` and the tab navigates to the signin address for the original channel with `next=` the home page carrying `p9bd=r` + the run tokens. That page (`runRestore`) re-reads the switcher: `restore='ok'` or `restore='failed'` (+ `restoreMsg`), then the group is finished and the popup's orchestrator moves on after the usual gap. A stale `pending` becomes `failed`. The popup shows a red row "Container 'X': could not switch back to channel 'Y'" for a failed restore. **Limits.** Unverified against the real YouTube (the JSON shape and the `/signin?…pageid=` redirect are from public markup and memory — the mock reproduces them). One page load per switch (~10–15 s); only the channels of the signed-in Google account of each container; no DOM-menu fallback: if YouTube changes the switcher the list shows "no channel list" and likes still work from the container's active channel; Stop mid-run can leave a non-original channel active.

5. Panel 9 requirements / assumptions

- **Panel 9 ≥ 2.3.9 for container selection.** Electron's extension subset has no `chrome.tabs`, so an extension cannot list containers or open a tab in one. 2.3.9 adds two methods to the `window.panel9ext` bridge (`src/extbridge.js`, `src/ext-bridge-preload.js`, `src/main.js` `listContainers` / `openTabInContainer`), allow-listed **only** for ids matching `/(^|-)panel9-bean$/` and only from the popup's main frame:
  - `containers()` → `{profile:{id,name,isPrivate}, active, containers:[{id,name,color,builtin,signedIn,checked,people:[names]}]}` (no e-mail addresses, cookies or partition names).
  - `openTabInContainer({containerId, url})` → opens `url` in a new tab of that container of the **same profile window** (`createTab(ctx, url, 'desktop', {container})`). `url` must match

`^https://www.youtube.com/watch?…` (≤ 400 chars, no whitespace/fragment); unknown container ⇒ error; at most 30 opens per 60 s.
  - `openTabInContainer` also accepts `https://www.youtube.com/?<query>` (channel discovery) and `closeTab({tabId})` closes only tabs this bridge opened itself (tracked per window; refuses user tabs and other extensions).
  - Also in 2.3.9 (needed for reliability): `extensions.js` storage-mirror fixes — `_push` sends only deltas, echoes of its own pushes are ignored for 4 s, `_onSnapshot` merges per key (previously a stale whole-snapshot could revert a newer value, which lost Bean's run record/selection).
  - Patch + tests: `panel9-2.3.9-bean-bridge.patch` (next to this file), test `test/bean-bridge-checks.js`.
- The popup feature-detects the bridge. On an older Panel 9 (no `window.panel9ext.containers`, or the call throws) it hides the picker, shows "requires Panel 9 2.3.10" and behaves as a single-container liker: `window.open` ⇒ `openExtensionPopup()` ⇒ `createTab(ctx, url, 'desktop', {container: active tab's container})`.
- Works in every container/profile because content scripts and storage exist in every session. Storage writes are visible to the other sessions (mirror) but only the tab holding the batch id + container id acts.
- There is **no min-version field** for the store or the manifest: `panel9_min_version` would be an unknown manifest key. The requirement (**Panel 9 2.3.10+**) is stated in the manifest description, the listing text, the README and the in-popup fallback note. Publish Panel 9 first, then this extension.

6. Tests (also see §8)

- `node tests/unit.js` — 33 checks (parsing, pacing, planActions/caps/estimate, switcher parsing, switch URLs, manifest/static rules, the single allowed `fetch`).
- `cd /workspace/mobile-browser && xvfb-run -a -s "-screen 0 1440x1000x24" npx electron --no-sandbox /workspace/panel9-bean/tests/e2e.js` (set `P9_DIR` for another Panel 9 tree; it needs the 2.3.9+ bridge for sections 10–18). It loads the real YouTube Focus + a copy of Bean named "Panel9 Bean" (so its id matches the bridge allow-list) into a test Panel 9 (temp user data) and drives the popups like a user against `tests/mock-youtube.js` (www.youtube.com → local HTTPS mock via `--host-resolver-rules`; each container's session gets a cookie `acct=<name>` that stands for "its Google account", and the mock keeps likes per account). Covers the 1.0 cases (comment-only off/on, link formats, replies, already-liked untouched, not-found, Stop, non-registering like = one click, late "already", comments off, slow, signed out, yt-focus disabled) and: picker lists containers, All/None, remembered selection, links × containers likes from the right account in the right container's tab (container-major order, one tab per container, pauses between links and between containers), rerun = all "already", signed-out container skips only itself, nothing ticked = current container, deleted container dropped from the selection, 200 cap + warning, Stop between containers, comment-only off stops before the second container's tab opens, and the no-bridge fallback. Channel sections: Refresh channels with accounts of 1, 2 and 3 channels (cache, no e-mail/cookie, discovery tabs closed), the tree and All/None, a run across channels (strictly sequential, each like from the intended channel, an already-liked comment on one channel only, original channels restored), a switch that cannot be verified ("Could not switch to channel X", no like), a failed switch-back, the cap with channels, and a failed refresh keeping the old list. The mock answers `/getAccountSwitcherEndpoint` and `/signin?pageid=…&next=…` per account (cookies `acct`, `chan`).
- Panel 9 side: `test/bean-bridge-checks.js` (`npm run test:bean`) plus the existing suites.
- UNVERIFIED: a real signed-in YouTube session (markup reproduced from public element names/ids: `ytd-comment-view-model`, `#like-button button[aria-pressed]`, `#published-time-text a[href*=lc=]`, `#more-replies`), several real Google accounts, Windows, m.youtube.com.

7. Maintenance

- YouTube markup: all selectors are at the top of the "actual logic" section of `content.js` (`COMMENT`, `findComment`, `likeButton`, `pressed`, `signedState`, `signInDialog`). If a selector breaks the failure mode is "Comment not found"/"unknown-state" with no click.
- Keep storage keys (`p9bn.*`) stable; add new keys with defaults.
- Package: zip with `manifest.json` at the root; `manifest.version` must equal the store version.
- Publish order: **Panel 9 2.3.9 first** (it contains the bridge), then the extension (see the report / README of the release for the exact commands; first release needs the listing fields): `bash -lc '/workspace/appstore/bin/publish.sh panel9-bean 1.0.0 /workspace/panel9-bean-1.0.0.zip "First release: …" --type extension --target panel-9 --platform any --name "Bean" --short-description "…" --description "… Requires Panel 9 2.3.9 …" --category Extensions --developer Oluchi --icon /workspace/panel9-bean/store-icon-512.png --screenshot`

`/workspace/panel9-bean/shots/popup-5-container-picker.png --source /workspace/panel9-bean --arch /workspace/panel9-bean/ARCHITECTURE.md'`

8. 1.1.0 additions (VPN rotation, ledger, named API)

- **Files**: `ext11.js` (all 1.1.0 UI + API/ledger/VPN glue, `self.B11`), `lib/vpnrot.js` (Rotator: probe/regions/preflight/rotate), `lib/apiclient.js` (server client, install identity), `lib/ledger.js` (sharded ledger), `API.md` (the external API), `panel9-bean-vpn-bridge.patch` (Panel 9 side, **not part of the extension zip**).
- **Roles**: the same `popup.html` runs as `popup` or as the hidden `runner` (`popup.html?runner=1`, created by the patched Panel 9 while `p9bn.api` is enabled). A batch has `owner` (`popup` | `runner`) and, for API runs, `job`; a page only drives batches it owns. The executor (runner if its heartbeat `p9bn.runner` is < 12 s old, else the popup) polls the server, runs jobs and is the **single writer of the ledger and of its server mirror**. The popup only adds to the ledger on import/restore and asks the runner (`p9bn.cmd` / `p9bn.cmdres`) to clear/prune.
- **Storage keys, one writer each** (Panel 9 2.3.9 merges `chrome.storage.local` per key across sessions): `p9bn.api` popup UI; `p9bn.apistat.popup|runner` that page; `p9bn.runner` runner heartbeat; `p9bn.vpnreq` the YouTube tab; `p9bn.vpn`, `p9bn.vpnptr` the batch owner; `p9bn.led.<hh>` (256 shards by hash of the comment key), `p9bn.led.ch` (channel table, union-merged), `.meta`, `.out` (pending mirror, ≤5000), `.rec` — executor only.
- **VPN handshake**: before each like the content tab writes `p9bn.vpnreq {rid,bid,idx,cid,at}`; the owner switches location through the Rotator (`panel9.vpn({op:'regions'|'state'|'connect'})` — the only three ops Bean may call; fields whitelisted and clamped by Panel 9), polls `state` until `connected` (bounded), confirms `exitIp` differs (never `publicIp`, which returns the real address), then answers `p9bn.vpn {rid,stage,ok,code,msg,location,ipChanged}`. The tab waits ≤25 s for the ack, ≤260 s overall; a failed switch = no like. A bypass/proxy container gets `vpnSkip` (flagged) or `vpn-bypass` (skipped).
- **Ledger**: key = canonical video id + comment id (reply = `PARENT.REPLY`), value = set of channel keys (`h:@handle` | `p:<pageId>` | `c:<containerId>`); kind `l` liked / `a` already. Planning (`planApi`/manual) consults it before any page is opened.
- **Server** (`/workspace/appstore/server/bean_api.py`, deployed to `/opt/appstore/server`, data `/opt/appstore/bean/bean.sqlite`, WAL): external `/api/bean/v1/*` (key `bean_…`), agent `/api/bean/agent/*` (install token `bit_…`); only SHA-256 hashes stored; token-bucket rate limits; per-install queue, long-poll (≤25 s), 120 s lease, sweeper (queue expiry 24 h, results 7 d), idempotency key. Tests: `tests/server_test.py` (16).
- **Tests**: `tests/unit.js` (33 + 14 from `unit11.js`), `tests/e2e.js` (the 1.0.0 scenarios + 1.1.0 "old Panel 9" scenario), `tests/e2e11.js` (real Bean in a patched Panel 9 at `/tmp/p9-bean11`, mock YouTube, mock `expressvpnctl`, real `bean_api` on a throw-away DB: `E2E_SKIP12=1` jumps to the API part), Panel 9 side `test/bean-vpn-checks.js` (in the patch).
- **TAB RULE (at most ONE Bean tab at any time)**: `popup.js` — `closeTabId()` / `closeRunTabs()` use the `closeTab` bridge (only tabs Bean opened itself; ids are tracked by Panel 9). The tab id of a container's tab is stored in the run record (`groups[].tab`) so any Bean page (popup or the background runner) can close it. `advance()` (a) closes a container's tab as soon as its group is `done` (all items final, channel restored), (b) before opening the next container's tab closes everything of Bean's that is still open (belt and braces against stale copies of the record), (c) closes every tab when the run is over (`done` / `stopped` /stale → `stopped`), on Stop, on Clear, and on popup/runner start (`init`) for runs left behind. Discovery (`refreshChannels`) opens one tab per container, **awaits** the close before the next, records the tab id in `p9bn.disc`, refuses to start while a run is in progress, and an API job waits (≤90 s) while discovery is running. Without `closeTab` (older bridge) nothing can be closed: the single `window.open` tab of the no-bridge fallback is left as in 1.0.0. Test: `tests/tabtrack.js` samples the tab list every 40 ms; `e2e.js` and `e2e11.js` assert max-at-once ≤ 1 for UI runs, API jobs, discovery, Stop, and that none is left at the end; the Panel 9 openTab rate limit is 30 per minute.
- **Publish** (needs Oluchi's approval; Panel 9 release containing the patch first): see the report; command pattern as in §7 with version `1.1.0` and `/workspace/panel9-bean-1.1.0.zip`.

Appendix A — Bean API reference (copy of API.md; also served by the store's architecture endpoint)

Bean API (Bean 1.1.0)

Submit comment-like jobs to **one specific Bean install** over HTTPS, and read the results.

> ***One key = one install.*** *An API key is created inside a Bean install ("Create API" + a name tag) and drives* ***only that install*** *— its saved containers/channels, its VPN locations, its ledger. A key can never read, cancel or queue anything on another install. If you run Bean on three PCs you create three APIs (three names, three keys) and talk to each one separately.*

Base URL: `https://api10.norhartpro.com/api/bean/v1`

Everything is JSON (`Content-Type: application/json`), UTF-8. Times are ISO-8601 UTC (`2026-10-30T19:04:11Z`).

1. How it works (30 seconds)

```
your app ──HTTPS──► api10.norhartpro.com/api/bean/v1/… ◄──long-poll── Bean (Panel 9 on the PC)
   POST /jobs  ─► queued ──────────────────────────────► Bean claims it, likes from its saved channels,
reports each like
   GET  /jobs/<id> ◄──────────── per-like results ◄──────
```

- Bean connects **outbound** to the server (no inbound port, no tunnel to the PC). The server only queues.
- Bean runs a job with the containers/channels, pace and VPN locations **saved in Bean**, and the **ledger**: a channel is never used twice on the same comment.
- Jobs only run while **YouTube Focus "Comments only" is on** in Panel 9; otherwise the job fails with `focus_off` .
- The PC must be on, Panel 9 running (with the Bean-VPN-bridge release of Panel 9 Bean runs jobs in a hidden background page; on an older Panel 9 Bean's window must stay open). `GET /install` tells you whether the install is `online` .

2. Authentication

```
Authorization: Bearer <your-bean-key>
```

(`X-API-Key: bean_…` is accepted too.) The key is shown **once** in Bean when you press *Create API*; afterwards Bean shows it masked (`bean_AbCd…wXyZ`). *Regenerate* replaces it (the old key stops working immediately); *Revoke* kills it. Only a SHA-256 hash is stored on the server. Keep the key in a secret store, never in a repository or a log.

3. Concepts

Comment link — any YouTube link that carries a comment id: `https://www.youtube.com/watch?v=VIDEOID&lc=COMMENTID` , `https://youtu.be/VIDEOID?lc=COMMENTID` , `…/shorts/VIDEOID?lc=…` . A **reply** has an id `PARENT.REPLY` (the `lc` value of a reply link); a reply is its own comment. A link without `lc=` is rejected (`invalid_link`). Parameters other than `v` / `lc` are ignored; the same video+comment given twice in one request → `duplicate_link` .

Channel — a YouTube channel (a profile) that Bean can like from; the install's *saved channels* are the containers/channels ticked in Bean. One like = one channel.

Capacity of a link = `channels_saved − channels_used_on_this_comment − reserved_by_queued_jobs` .

- `channels_used` counts saved channels the ledger already recorded as used on that exact comment (liked it, or found it already liked). A channel is never used twice on a comment, ever.
- `reserved_by_queued_jobs` are likes promised to jobs that are still queued/running for the same comment, so two jobs never promise the same channel twice.
- `max_likes` is what you can still ask for.

Quantity — likes wanted for a link (integer 1–200).

- **omitted → all available** (`max_likes` at submit time; if that is 0 the request fails with `no_capacity`).
- above capacity → **rejected** with `422 quantity_exceeds_capacity` (default), or **clamped** to the maximum when you send `"over_quantity": "clamp"` (the link then shows `"clamped": true`).

4. Endpoints

`GET /health` (no auth)

```
{"ok": true, "service": "bean-api", "version": 1, "time": "2026-10-30T19:04:11Z"}
```

`GET /install` — which install this key drives

```
{
  "install_id": "bi_hGuemZ7-0Y2ebLjo0N6y4wvL", "name": "studio-pc",
  "online": true, "last_seen_at": "2026-10-30T19:04:09Z", "seconds_since_seen": 2,
  "bean_version": "1.1.0", "created_at": "2026-10-30T18:00:00Z",
  "api_key_hint": "bean_AbCd…wXyZ", "key_created_at": "2026-10-30T18:00:00Z",
  "queue": {"queued": 0, "running": 1},
  "saved_channels": 4, "youtube_focus": "on",
  "vpn": {"enabled": true, "locations": 3}, "ledger_entries": 211,
  "config_updated_at": "2026-10-30T19:03:55Z"
}
```


`online` = Bean polled within the last ~45 s. `youtube_focus` : `on` | `off` | `unknown` (unknown = no YouTube tab answered; jobs still try). If `online` is false jobs queue and wait (up to 24 h, then `expired`).

`GET /capacity?link=<url-encoded link>&link=…` or `POST /capacity {"links": [...]}`


```
{"install_id": "bi_…", "name": "studio-pc", "online": true, "as_of": "2026-10-30T19:03:55Z",
 "capacity": [{
   "vid": "dQw4w9WgXcQ", "lc": "UgxABC…", "link": "https://www.youtube.com/watch?
v=dQw4w9WgXcQ&lc=UgxABC…", "is_reply": false,
   "channels_saved": 4, "channels_used": 1, "reserved_by_queued_jobs": 0, "max_likes": 3}]}
```


Up to 20 links. `as_of` is when Bean last synced its channel list (the answer is as fresh as that).

`POST /jobs` — submit a job

Request:


```
{
  "links": [
    {"link": "https://www.youtube.com/watch?v=dQw4w9WgXcQ&lc=UgxABC…", "quantity": 3},
    "https://youtu.be/9bZkp7q19f0?lc=UgxDEF….REPLY1"
  ],
  "over_quantity": "reject",
  "note": "campaign 12",
  "idempotency_key": "order-8841"
}
```


- `links` — 1–20 items: a string, or `{"link": "...", "quantity": n}` . (`{"link": "..."}` at top level is accepted for one link.)
- `over_quantity` — `"reject"` (default) or `"clamp"` .
- `note` — free text ≤120 chars, echoed back. `idempotency_key` (or header `Idempotency-Key`) — resubmitting the same key returns the original job (HTTP 200, `"idempotent_replay": true`) instead of a duplicate.

Response `202 Accepted` (+ `Location: /api/bean/v1/jobs/<id>`):


```
{
  "job_id": "job_Qnkfw9fzFV_AQvyL", "status": "queued",
  "created_at": "…Z", "started_at": null, "finished_at": null, "expires_at": "…Z",
  "note": "campaign 12",
  "links": [
    {"n": 1, "link": "https://www.youtube.com/watch?v=dQw4w9WgXcQ&lc=UgxABC…", "vid": "dQw4w9WgXcQ",
"comment_id": "UgxABC…", "requested": 3, "accepted": 3, "clamped": false, "max_at_submit": 4},
    {"n": 2, "link": "…", "vid": "9bZkp7q19f0", "comment_id": "UgxDEF….REPLY1", "requested": null,
"accepted": 4, "clamped": false, "max_at_submit": 4}
  ],
  "progress": {"total_likes": 7, "done": 0, "liked": 0, "already_liked": 0, "failed": 0, "skipped": 0},
```

```
  "results": [], "error": null, "warnings": [], "cancel_requested": false, "install_online": true
}
```

All-or-nothing: if any link is invalid / over capacity (reject mode) nothing is queued and you get a 4xx with details (see §6).

`GET /jobs/{job_id}` — status + per-like results

```
{
  "job_id": "job_Qnkfw9fzFV_AQvyL", "status": "running",
  "created_at": "…Z", "started_at": "…Z", "finished_at": null, "expires_at": "…Z", "note": null,
  "progress": {"total_likes": 7, "done": 3, "liked": 2, "already_liked": 0, "failed": 1, "skipped": 0},
  "links": [ … as above … ],
  "results": [
    {"n": 1, "seq": 0, "status": "liked",  "code": "", "message": "", "container": "Channel A",
"channel": "Alice", "handle": "@alice", "at": 1761850000.3,
     "vpn": {"location": "UK - London", "ip_changed": true, "bypassed": false}},
    {"n": 1, "seq": 1, "status": "skipped", "code": "ledger-used", "message": "Channel already used on
this comment", "container": "Channel A", "channel": "Alice Cooking", "handle": "", "at": 1761850004.1},
    {"n": 1, "seq": 2, "status": "failed", "code": "vpn-stuck", "message": "VPN stayed on “connecting”
— not liked", "container": "Channel B", "channel": "Bob", "handle": "@bob", "at": 1761850020.9}
  ],
  "error": null, "warnings": [], "cancel_requested": false
}
```

`results[]` is one row per like attempt (`n` = link number, `seq` = attempt number within the link). `status` ∈ `liked` , `already` (that channel had already liked it — not clicked), `failed` , `skipped` . `code` is Bean's reason for non-liked rows, e.g. `ledger-used` , `not-signed-in` , `not-found` , `comments-off` , `unavailable` , `click-failed` , `switch-failed` , `open-failed` , `interrupted` , `stopped` , `not_started` , `no_capacity` , `vpn-failed` , `vpn-stuck` , `vpn-noip` , `vpn-unavailable` , `vpn-bypass` . `vpn` appears when the install rotates ExpressVPN locations; `bypassed: true` means that container is set to *Outside VPN*, so the like was made from its own address (a warning is also added to `warnings[]`). `ip_changed` is `true` when Bean read the exit address before and after the switch and they differ, or `null` when there was no readable exit address to compare with before the switch (the new exit address was still confirmed non-empty and a like is never made after a failed or unconfirmed switch); it is never `false` on a liked item. Results are appended as likes happen — poll to watch progress. Likes are never retried automatically (a retry could un-like).

`GET /jobs?status=running&limit=20` — recent jobs (newest first, ≤100)

```
{"jobs": [{"job_id": "job_…", "status": "completed", "created_at": "…Z", "finished_at": "…Z",
"progress": {…}, "error": null, "warnings": [], "note": null, "expires_at": "…Z", "started_at": "…Z",
"cancel_requested": false}]}
```

`POST /jobs/{job_id}/cancel` — optional cancel

Queued → `cancelled` immediately. Running → `"cancel_requested": true` ; Bean stops after the like in progress, likes already made stay made, the rest are not done, final status `cancelled` . A finished job answers `200` with a `message` saying there was nothing to cancel.

5. Job lifecycle

status	meaning
`queued`	waiting for Bean to poll (install offline, or busy with an earlier job — jobs of one install run **one at a time, in order**)
`running`	Bean claimed it and is liking
`completed`	every requested like was made (`liked` + `already` = total)
`partial`	finished but some likes did not happen (see `results[]` / `error`)

`failed`	nothing could be done — see `error` (e.g. `focus_off`) — or Bean vanished mid-job without results
`cancelled`	you cancelled it
`expired`	nobody claimed it for 24 h (install offline)

`error` on a job: `{ "code": "...", "message": "..." }` — `focus_off` (YouTube Focus "Comments only" is off: switch it on in Panel 9 and resubmit), `focus_missing`, `vpn_unavailable` (ExpressVPN needs Panel 9 run as administrator and the ExpressVPN app running and signed in), `no_channels_saved`, `no_capacity` (the ledger/channels changed since you submitted), `no_bridge` (Panel 9 too old), `busy`, `interrupted`, `agent_lost` (Panel 9 closed or offline mid-job; likes already made are listed), `stopped`, `internal`. Finished jobs and their results are kept **7 days**, then deleted (`404 job_not_found`).

6. Errors

HTTP status + `{"error": {"code": "...", "message": "...", "details": {…}}}` .

HTTP	code	when
400	`invalid_json`, `invalid_request`, `invalid_link`, `duplicate_link`, `too_many_links`, `invalid_quantity`	malformed request (`details.index` = which link)
401	`invalid_api_key`	missing / wrong / revoked / replaced key
404	`job_not_found`, `not_found`	unknown job (or a job of another install), unknown route
409	`no_channels_saved`	the Bean install has no channels selected yet
413	`payload_too_large`	body > 1 MB
422	`quantity_exceeds_capacity`	a quantity is above `max_likes` (use `over_quantity: "clamp"`); `details.problems[]` lists `{n, link, requested, max_likes}`
422	`no_capacity`	a link has no unused channel left; nothing queued
429	`rate_limited`, `queue_full`	slow down (see `Retry-After`); ≥25 queued jobs
403	`install_revoked`	(Bean-side) the API was revoked

7. Rate limits and polling

- **120 requests/min per key**, **10 job submissions/min per install**, **25 queued jobs per install**, 20 links and 200 likes per link per request. Headers `X-RateLimit-Limit`, `X-RateLimit-Remaining` ; on `429` honour `Retry-After` (seconds).
- Poll `GET /jobs/{id}` every **5 s** while `running` , every **15–30 s** while `queued` ; stop at `completed|partial|failed|cancelled|expired` . Likes are paced by Bean (2–10 s each by the pace saved in Bean, longer between channels/containers), so a 50-like job takes minutes — do not poll faster than once per 3 s.
- Ask `/capacity` before submitting if you need to size a job; submit with `over_quantity: "clamp"` to take whatever is left.
- Use `idempotency_key` when your caller may retry a POST.

8. Snippets

curl

```
export BEAN_KEY='bean_…'            # from Bean → API → Create API (shown once)
B=https://api10.norhartpro.com/api/bean/v1
curl -s $B/install  -H "Authorization: Bearer $BEAN_KEY"
curl -s "$B/capacity?link=$(python3 -c 'import urllib.parse,sys;print(urllib.parse.quote(sys.argv[1]))'
'https://www.youtube.com/watch?v=dQw4w9WgXcQ&lc=UgxABC')" -H "Authorization: Bearer $BEAN_KEY"
curl -s -X POST $B/jobs -H "Authorization: Bearer $BEAN_KEY" -H 'Content-Type: application/json' \
  -d '{"links":[{"link":"https://www.youtube.com/watch?
v=dQw4w9WgXcQ&lc=UgxABC","quantity":3}],"over_quantity":"clamp"}'
```

```
curl -s $B/jobs/job_XXXX -H "Authorization: Bearer $BEAN_KEY"
curl -s -X POST $B/jobs/job_XXXX/cancel -H "Authorization: Bearer $BEAN_KEY"
```

JavaScript (Node 18+ / browser)

```
const B = 'https://api10.norhartpro.com/api/bean/v1', KEY = process.env.BEAN_KEY;
async function bean(path, body) {
  const r = await fetch(B + path, { method: body ? 'POST' : 'GET', headers: { Authorization: 'Bearer '
+ KEY, 'Content-Type': 'application/json' }, body: body && JSON.stringify(body) });
  const j = await r.json();
  if (!r.ok) { const e = new Error(j.error.code + ': ' + j.error.message); e.code = j.error.code;
e.details = j.error.details; throw e; }
  return j;
}
const job = await bean('/jobs', { links: [{ link: 'https://www.youtube.com/watch?
v=dQw4w9WgXcQ&lc=UgxABC', quantity: 3 }], over_quantity: 'clamp', idempotency_key: 'order-8841' });
let j = job;
while (['queued', 'running'].includes(j.status)) { await new Promise((r) => setTimeout(r, j.status ===
'running' ? 5000 : 15000)); j = await bean('/jobs/' + job.job_id); }
console.log(j.status, j.progress, j.results.map((x) => [x.status, x.channel, x.code]));
```

Python 3 (requests)

```
import os, time, requests
B = "https://api10.norhartpro.com/api/bean/v1"
H = {"Authorization": "Bearer " + os.environ["BEAN_KEY"]}

def bean(method, path, **kw):
    r = requests.request(method, B + path, headers=H, timeout=30, **kw)
    j = r.json()
    if not r.ok:
        raise RuntimeError(f'{j["error"]["code"]}: {j["error"]["message"]}')
    return j

job = bean("POST", "/jobs", json={"links": [{"link": "https://www.youtube.com/watch?
v=dQw4w9WgXcQ&lc=UgxABC", "quantity": 3}],
                                  "over_quantity": "clamp", "idempotency_key": "order-8841"})
while job["status"] in ("queued", "running"):
    time.sleep(5 if job["status"] == "running" else 15)
    job = bean("GET", "/jobs/" + job["job_id"])
print(job["status"], job["progress"])
for x in job["results"]:
    print(x["n"], x["status"], x["channel"], x["code"])
```

9. Notes for integrators

- Check `GET /install` first: `online`, `youtube_focus` and `saved_channels` tell you whether a job can run now.
- A like only counts once Bean saw it registered on YouTube (`liked`); `already` means the channel had already liked that comment.
- VPN: when the install has ExpressVPN locations saved, Bean switches to the next location before every like and only likes after it verified the exit address changed. A failed switch means **no like** for that item (`vpn-failed` / `vpn-stuck` / `vpn-noip`).
- Reinstalling Bean: the ledger is mirrored to the server; linking the saved key (*Bean → API → I already have a key*) restores the same install and its ledger.
- Tabs: Bean never has more than one of its own tabs open while it works through a job (containers are processed one after another and each tab is closed before the next one opens).

- No webhooks: poll. The API never exposes YouTube sessions, cookies, container names beyond what is in `results[]`, or any credential.

1.1.0 final — exact-id rule for Panel 9 2.3.10 (VPN bridge)

- Shipped 2.3.10 `beanVpn connect` accepts ONLY `smart` (case-insensitive) / empty, or an id that is exactly in `evpn.regions()`; Helper2's hardening replaced the old `/^smart/i` prefix test. Return shapes unchanged: `regions` → `{regions:[{id,name,cc}]}`; `state` → `{ok,bean:1,vpn:{state,location,reason,message,exitIp},routes[]}`; `connect` → `{ok,accepted}`.
- Bean: `Rotator.regions()` keeps `id` as given (no fallback to name) and drops ids/names starting with "smart" (Smart Location is never a rotation target). Before every `connect`, `Rotator.rotate` checks that `loc.id ===` an id in the current list (refreshing once); otherwise `vpn-badloc` and NO connect call. A list that cannot be read is `vpn-unavailable`.
- Saved locations (`p9bn.settings.vpn.locs`): `V.migrateLocs(saved, regions)` runs from `ext11.ensureExactLocs` (before a run, before "Test next switch", before a VPN request, and when the locations load in the UI): exact ids are kept; a non-exact value that matches exactly one region by id/name (trim + case-insensitive) is rewritten to the exact id; everything else (including `smart…`, ambiguous labels, junk) is dropped and `settings.vpn.note` shows a UI note. If all locations were dropped the run is refused (`vpn_unavailable`), never silently run from the user's own connection. The note is cleared when the user saves locations again.
- `noip: true` on the preflight probe and on the wait-for-connected polls; the exit address is read only for the baseline before a switch and after "connected at the right location" (Panel 9 caches it 3 s). `publicIp` is never read.
- Tests: `tests/unit11.js` mock enforces the exact-id rule (rejects `smart-location`, `Smart Location`, `SMART`, case/space variants, non-strings); e2e11 runs against the released 2.3.10 tree (`P9_DIR=/workspace/mobile-browser`), whose own `beanVpn` enforces it against the mock `expressvpnctl`.

9. 1.1.1 — click safety, channel safety, run log (from real 1.1.0 use)

Reported live: a comment un-liked by a second click; Bean switching into a channel nobody ticked; 3 likes recorded but only 1 visible on YouTube; a run that kept acting after "Done"; 17 of 20 items failed with no visible reason.

- **content.js `clickOnce` is the only place a like is clicked**, in this order: single-flight lock `p9bn.lock` (write → 260 ms → read back) → marker `p9bn.clk.<hash(vid|lc|channelKey)>` must not exist → ledger shard (`P9BEAN_LEDGER.storedHas`, one shard) must not list the channel → **fresh channel re-check** (`getAccountSwitcherEndpoint`, active == item.chid) → every copy of the comment reads "not pressed" → marker **written and read back** → one `click()` → aria-pressed must turn true within 6 s → must still be pressed 3.2 s later → the item is parked as `verify` and the page **reloads**; the fresh page (same code, `verifyMode` because the marker exists) reads the state with `stableState` and only then reports `liked` (`like-not-kept` when it is gone; the marker is dropped then, so a later run may retry).
- `stableState(lc)`: samples every 300 ms; "liked" the moment any copy shows pressed (aria-pressed true OR an "Unlike..." label); "unliked" only when every copy reads false, the (value, button element) signature did not change for 2.5 s, no aria mutation for 0.9 s, document complete and ≥5 s since navigation start; unreadable → `unknown-state` (never clicked).
- **Only one run logic per page**: `<html data-p9bn-inst>` claim. Tests inject a duplicate with the `p9bn-test-dup` event (only on mock pages that set `data-p9bn-test="dup"`).
- **Fence**: `patchNow` never writes once the stored run is not `running` (sets `fenced`, the tab stops and `retire()`s to about:blank); popup `heal()` remembers a run's final state and repairs a late "running" write; ext11 aborts a VPN rotation (`opts.stop`) when the batch is no longer running.
- **Restore** is `batch.restoreBack` (setting `restoreBack`, default false). `normalizeSel` no longer adds channels.
- **Run log**: items carry `lg: [[ms, step, detail]…]` (≤60); `L.runBrief/logText` build the text/JSON; popup archives each finished run to `p9bn.runs` (last 5). No IP/cookie/key is ever logged: only "exit address changed: yes/no".
- New result codes: `like-not-kept`, `wrong-channel`, `click-sent-before`, `click-unconfirmed`, `busy-lock`, `marker-failed` (all with hints in `L.HINTS`).
- Tests: `tests/e2e-safety.js` (R1 late hydration, R2 duplicate injection, R3 reload after click, R4/R5 revisit with stale `queued` item / without the marker, R6 unselected channel + restore option, R7 no auto-tick, R8 rejected like, R9 channel flips before the click); mock-youtube scenarios `LATEHYD`, `SLOWTOG`, `DUPINST`, `RELOADAF`, `REJECTLK`, `FLIPCHAN`; the mock server now decides what a click does (a click on an already-liked comment is an UN-LIKE) and records the server-side channel at click time.
- **First-seed guard** (Panel 9 race, fix due in 2.3.16): `popup.js` tracks `lastWriteAt` (every `sset`) and `advance()` waits until 2 s have passed before opening a container's first tab; `clickOnce` re-reads `p9bn.batch` and refuses to click unless it

is this run and `running`.
- Tab-opened run record re-read already waited up to 30 s for the matching record; the extra re-check right before the click covers a stale first read.

9.1 Audit of real-world risks (1.1.1)

Fixed in 1.1.1: unhydrated like state read (H), no click marker / ledger check at click time (H), duplicate content-script instances (H), unfenced late writes after Done (H), restore-to-original switching into an unticked channel (H), channel auto-tick (M), optimistic `aria-pressed` recorded as liked (H), wrong active channel at click time (H), VPN rotation continuing after a run ended (M), slow VPN no-answer wait (L), refusal messages overwritten by re-render (L), no visible failure reasons (M). Still open (not fixed — needs real YouTube data or Panel 9 work):

- **M** Real-YouTube DOM drift: `aria-pressed` /"Unlike" label detection, the channel-switcher JSON variants and the sign-in redirect were only tested against the mock; the run log now records enough (steps, codes) to diagnose.
- **M** A like that was sent but never verified (popup closed in the 3 s hold, tab killed) leaves a marker without a ledger entry: the next run only verifies that comment (`click-sent-before`) and will not click again — safe, but needs a manual look if YouTube dropped it.
- **M** Hidden/background tab timer throttling in Chromium can stretch the 3.2 s hold and the stable-state waits; waits are wall-clock based so this makes runs slower, not wrong.
- **M** Rate limiting / "try again later" from YouTube is not detected as its own code (shows as `like-not-kept` or `click-unconfirmed`).
- **L** `chrome.storage` mirror is last-writer-wins; Bean defends with known/finSeen/healFin and the fence, but two Bean windows at once are not supported (single-flight lock only covers clicks).
- **L** API queue/lease: a runner crash mid-job leaves a lease until it expires (server side); jobs are not resumed.
- **L** ExpressVPN edge cases: exit address unchanged after "connected" (retried, then `vpn-stuck`); the exit address is never logged.
- **L** Panel 9 restart during a run: the run ends as "interrupted"; markers keep it from double-clicking on the next run.
- **L** Markers older than 60 days are pruned; ledger caps 30 000 comments / 3 000 channels.
- **L** Per-link "possible" rows need the ledger loaded; before that the panel says "Ledger still loading".

10. 1.1.2 - reload-safe runs, tab registry, VPN placement, speed (from the real 1.1.1 log)

Liveness. `L.staleAt(b) = max(b.updated + 45 s, b.graceUntil)`. `content.js` announces `graceUntil = now + 120 s` before every navigation it starts (channel switch, verify reload, next item, restore) and `patchNow` clears it on the next normal write; `popup.js` announces 75 s (`OPEN_GRACE_MS`) when it opens a tab. The popup/runner only treats a group as lost when the run is stale. **Ending a lost group (`endItems`).** a running item that has `sent` or `verify` becomes status `liked` / code `liked-unverified` (so `recordFinal` puts it in the ledger and nothing ever clicks it again); a running item without them becomes `interrupted`; queued items become `not-started`. When nothing is left to open the run ends `done`. **Tab registry.** `p9bn.tabidx = [{id, run, cid, at}]`, written by the opener right after `openTabInContainer` (`regTab`; channel-reading tabs use run `disc`). `sweepTabs(keep)` closes every registered tab except those of the working group of a live run (`activeKeep`). Called in `advance()` on every pass, before opening a tab, on `stop()`, at run start and at popup init. `closeTabId` treats "not a tab opened by this extension" as already gone. **VPN.** `rotateNow(bid)` (shared by popup/runner) runs before a container's first tab opens (`vpnBeforeOpen`, heartbeat every 8 s); inside a tab the NEXT item's location is switched in the pace gap while the page is idle; `checkNow` is a read-only state check requested right before the click. Test: e2e11 asserts no connect falls inside any /watch, /signin or switcher in-flight window of the mock (`state.reqLog`). **Speed.** MutationObserver comment finder; `g.active` fast path (skip the slow channel-list read when the previous item verified the channel - the fresh `recheck` before the click is never skipped); `hideOthers` = `<style id="p9bn-focus-css">` with `:not(:has(a[href*="lc=..."]))` `display:none` (no DOM change); ledger flush 150 ms. A trial to shorten the verify page's `stableState` waits caused `like-not-kept` on late hydration (R3/R4/R5) and was reverted. A cheaper verify than a reload is not possible without cookies/tokens, so the reload stays (a like is only "liked" after a fresh page shows it). **Mock realism.** `tests/mock-youtube.js` `state.delay = {watch, switcher, signin, hops}` (number or function), a `/signin-hop` chain (the channel changes at the last hop) and `state.reqLog`. **Audit 10.1**

- **M** Ledger wire format has only `l` / `a` kinds; liked-unverified is stored as `l` locally. A distinct kind needs a server change (`bean_api.py`) - not done.
- **M** The 120 s / 75 s grace values are a trade-off: a tab that dies during a navigation costs up to that long before the run moves on.

- **L** Removing a ledger entry does not delete the copy the API server already has.
- **L** Cheaper-than-reload verification is not implemented (see above).

11. 1.1.3 - re-attaching after YouTube rewrites the URL (from the real 1.1.2 log)

Cause. `content.js` reads the run tokens (`p9bn` , `p9bc` , `p9bh`) from `location.href` once, at document_start. Real YouTube then calls `history.replaceState` and keeps only v/lc/t/list/index. 1.1.2's verify step used `location.reload()` , i.e. the stripped URL: no tokens, no `runBatch` , nothing logged; after `NAV_GRACE_MS` (120 s) `endItems` turned the item into `liked-unverified` and the rest of the container into `not-started` . **Fix.** (1) `goTo(url, b, item, cid, kind)` is the only navigation inside a run: it writes the hand-over, then `location.assign(L.watchUrl(...))` with all tokens; used for the channel switch (twice: with / without the channel list), the verify page and the next item. (2) `p9bn.bind = {run, cid, chid, vid, lc, kind, at, used?}` ; `adoptBind()` (only for a page WITHOUT tokens, only /watch with v and lc equal to the hand-over's, only within `NAV_GRACE_MS + 60 s` , once - `used` is written before the run starts) rebuilds the page info. Adoption also requires (read-only) that the run is `running` and still has a queued/running item for exactly that video, comment, container and channel. `sweepTabs(null)` removes it. (3) The click path is unchanged: a sent like leaves the marker `p9bn.clk.*` , so the resumed page is in verifyMode (reads, never clicks). (4) `orphan(why, b)` logs path + reason to `p9bn.diag` and to `batch.orphans` (shown in the run log) when a page cannot use its run. **Tests.** `mock-youtube.js` : `state.strip` (ms; rewrites the URL after load) and `state.stripSignin` (the sign-in chain's final redirect keeps only v/lc). Safety R18 (verify + next channel with URL rewriting) and R19 (also the sign-in chain). Pre-fix (1.1.2 tree): R18/R19 fail with `liked, not-started` after 135 s, as in the real log. **Audit 11.1**

- **M** Adoption cannot tell which Panel 9 container a token-less page is in (the content script has no tab id). The channel recheck, the stable-state check, the marker and the ledger still guard every click; a user who opens exactly that comment link in another container within the grace period could be adopted once. Accepted trade-off.
- **L** A hand-over older than 3 minutes is ignored.
- **L** If YouTube also removes `v` or `lc` the page cannot be matched; it is then not adopted (logged only when it carried tokens).

12. 1.1.4 - the channel switch, bounded waits, Pause / Resume (from the real 1.1.3 log)

Symptom. Real log: item 1 liked and verified; item 2 (the container's 2nd channel): `switch - attempt ... via YouTube's own switch address (no channel list read: the active channel was verified on the previous item) > open - page loaded` > about 66 s of nothing > `interrupted` . **Real mechanism (public sources).** `GET /getAccountSwitcherEndpoint` (`)]}'` prefix) lists the account's channels; each `accountItem.serviceEndpoint.selectActiveIdentityEndpoint.supportedTokens` has `accountSigninToken.signinUrl` (`/signin?action_handle_signin=true&authuser=N&next=...&feature=masthead_switcher&skip_identity_prompt=true[&pageid=<21 digits>]`), `pageIdToken.pageId` (brand channels only), `datasyncIdToken` , `accountStateToken` , `offlineCacheKeyToken` . The web UI navigates to that signinUrl. Sources: yt-dlp PR #682, namida issue #726, the sayaka-sh/spmp commit, meiro `harness_test.go` , YouTube Help "Switch between channels", Keyboard Maestro / Reddit threads. **Cause (moderate confidence, no real-YouTube data).** The 1.1.3 fast path skipped the list and navigated to a hand-made address (`/signin?...&next=`) without authuser/feature/skip_identity_prompt/pageid; real YouTube does not switch for it. The landing page then had no usable run (replaced page / hand-over used up / another runner still "owning" the item) and logged nothing, and the switcher read had no timeout. **Fix.** `readChannels` is bounded (AbortController 20 s, 3 attempts, 70 s race); the list is read before every switch and its own signinUrl is used (the `lib/links.js` fallback now at least carries authuser/feature/skip_identity_prompt); `skipRead` only when no switch is needed; the landing page logs path, readyState and how it found its run; takeover waits up to 25 s for a stale owner; `adoptBind` is rolling (max 6 adoptions, run running, item queued/running). A failed verify ends `switch-failed` . **Pause / Resume.** Record fields `pauseReq` (requested) and `pause {since, where}` (engaged); `events[]` for the log. `holdIfPaused` in `content.js` runs only after an item's final result is recorded, before the next item's VPN switch, before its navigation and before switching back; it idles with a heartbeat. The popup's `advance()` holds before opening the next container (no tab, no VPN) and after a VPN switch re-checks the request. `staleAt` and the popup's `RESUME_MS` treat an engaged pause as alive for 6 h (`PAUSE_MAX_MS`), after which the popup stops the run and logs it. A Pause that arrives while a VPN switch is already running lets that switch finish (never abort mid-load); the page then holds before loading. The tab stays open while paused (closing and reopening would add a navigation); between containers there are zero tabs. The API server drops unknown config keys, so no `paused` field exists server-side; API.md documents that a paused job stays `running` . **Tests.** Safety R21-R23 (switch address, reload after switch, hanging switcher), R24-R26 (Pause in a tab, Pause between containers, 6 h limit), unit11 static checks.

13. 1.1.5 - one button system (popup.css)

`.btn{display:inline-flex}` overrode the `hidden` attribute and the 1.1.2 `.btn` rule (min-height 40, padding 20) overrode `.btn.small`; the action row was `justify-content:flex-end`. 1.1.5 appends one block to `popup.css`: `[hidden]{display:none!important}`; `.btns` is a 2-column grid (Start and Clear span both columns; Pause|Resume and Stop are halves; `popup.js` hides `#go` while a run is going); `.btn.small` is 32 px with nowrap; `.srow` wraps whole buttons; `.logbar` is a 3-column grid; `.chead` is title + toolbar row; ledger rows wrap instead of ellipsis. Screenshots: `tests/shots.py` (`shots/before-1.1.4/`, `shots/after-1.1.5/`). No JS behaviour change except `els.go.hidden`. **Stale like after a switch.** `processComment` returns `stale-liked` for a liked first reading on a page that follows a switch (`item.tries >= 1 && !item.fresh`); `runBatch` then does one `goTo` of a cache-busted address (`p9bt`, kind `verify`) with `item.fresh` set, and the normal read runs (channel re-read included). Still liked on that fresh page: `already`, unless the same container's earlier item for this comment is `liked` in this run: then `identity-unclear` (no click, not in the ledger). Mock: `state.staleLikeAfterSwitch` (true: the first page after a switch is stale; 'persist': every page). Tests R27, R28, unit11.

13.1 1.1.5 - VPN request path and the ledger "already" clear

`vpnSwitch` (content.js) writes `p9bn.vpnreq` {rid, bid, idx, cid, check, at, n, fallback}. 1.1.4 gave up when no `p9bn.vpn` answer with that rid showed 'working' within 15 s (real log: item #20, `vpn-failed`). Now the tab re-writes the same rid every 4 s (a changed `at` fires `storage.onChanged` again), sets `fallback` after 20 s and fails after 45 s. `onVpnReq` (ext11.js) is entered from the storage event, from a 1.5 s poll of `vpnreq`, and from re-sends; `S.vseen` (rid + 'f') lets one evaluation run at a time, the owner (`batch.owner`) answers, a hidden runner answers only a `fallback` request, and a page that finds an answer for the rid already there does nothing, so two pages never switch the VPN for one request. `S.dropVreqUntil` is a test hook (the page ignores requests until then). The "never switch while a page loads" rule is unchanged: the tab only asks while no page is loading. `Ledger.kinds(vid, lc)` returns {l, a}; `Ledger.removeKind(vid, lc, 'a')` removes only the "already" entries of one comment (entry by entry through `remove`, so the mirror queue and recent list drop them too) and needs `flushAll()`. UI: `ledclearalready` bar in `renderLedRows` (only when the rows are one comment), command `remove-already` through `ledCmd` (routed to the hidden runner when it is the ledger writer).

13.2 1.1.5 calm restyle

The last block of `popup.css` ("1.1.5 calm") remaps the 1.1.1 `--bean-*` variables to Panel 9's `--acc*` tokens and overrides the old identity rules: no gradients, no outlined cards or inputs (fields and lists are `color-mix(--fg 5%, --surface)` fills), pill buttons (secondary `--surface-3`, primary `--acc`, danger `--danger-soft`), Panel 9's 18 px checkbox (data-URI check, no switches anywhere), hairlines at 40 % (`--rule`), 6 px progress bar, round status dots. Spec: `/workspace/shared/panel9-ui-style.md`. Icons: `tests/make_icons.py` (flat indigo tile + white bean). Screenshots: `shots/before-minimal/` (caramel) and `shots/after-minimal/`.

14. 1.1.6 - page identity, pause key, ledger comment clear

- **Identity source.** `content.js pageIdentity()` scans the inline `ytcfg.set({...})` scripts for `DELEGATED_SESSION_ID`, `DATASYNC_ID` (`pageId||userId`, personal `userId||`) and `SESSION_INDEX` (read-only). `links.identityVerdict(info, chid)` -> `match | MISMATCH | unreadable` (only the last 6 digits are logged). The switcher fetch (`readChannels`) is NOT an identity source: after a switch it can report the new channel while the page's session (the `X-Goog-PageId` of the like request) is still the old one.
- **Gates.** (1) after a switch the "verified" step needs fetch AND page identity; a disagreement repeats the sign-in navigation (`item.tries` 1..2), then `switch-failed`; (2) `skipRead` only when the page identity already matches; (3) `recheck` (right before the click) checks the page identity first; unreadable ytcfg fails closed. Limits: the masthead name is not in the initial data without opening the menu, so it is not compared.
- **Pause.** `p9bn.pause {bid, at}` (popup-only writer; `at:0` = resumed) is the truth; `batch.pauseReq` is a mirror (`L.applyPauseKey` heals it in `advance`, the held tab re-asserts `pause`). New hold points: before a channel switch, and at the start of an item that has no like marker (and in `processComment`, before the VPN/like). Never between click and verify.
- **Ledger.** `Ledger.removeComment(vid, lc)`; op `remove-comment` also removes the `p9bn.clk.*` markers of the comment and sets `ledgerVoid` on matching items of the current run record (`recordFinal` skips them).
- **Mock.** `ghostSwitch` (true | N): cookie `chan` (switcher) follows the switch, cookie `ident` (page ytcfg + likes) stays on the previous channel until the Nth attempt.

15. 1.1.6 - the identity module (lib/identity.js) and the switch state machine

One question, answered in one place: **which channel is this page really rendered for, and may Bean click now?** Everything that used to be ad-hoc (the switcher fetch, ytcfg scraping, "verified" messages) is replaced by `lib/identity.js` (pure, unit-tested, loaded in the content script before `content.js`).

Signals (each returns `match | mismatch | unknown` plus a plain-English phrase; only the last 6 digits of an id are ever shown):

1. *Page session* (`signalPage`): `ytcfg` `DELEGATED_SESSION_ID` / `DATASYNC_ID` (`<pageId>||<userId>`; the personal channel has no delegated id and `<userId>||`) = the `X-Goog-PageId` the like request is sent with. **Required.** `parseYtcfg` waits up to 6 s for slow ytcfg (`readIdentity`).
2. *Account menu* (`signalMenu`): the switcher's active entry, fetched fresh (`/getAccountSwitcherEndpoint`), compared by stable id (pageId / `main`), never by name or position. **Required.**
3. *Header picture* (`signalMasthead`): the masthead avatar compared with the target's picture in the menu. Used when readable; a readable picture of another channel is a conflict; unreadable is tolerated and said so in the log.

Verdict (`combine`): `confirmed` (page + menu agree, header agrees or unreadable) | `mismatch` (the page session is another channel) | `conflict` (signals disagree) | `unknown` (a required signal unreadable). Only `confirmed` can lead to a click.

State machine (`nextStep`, evaluated after every page load of an item that has a target channel): `proceed` when the menu lists the target as active AND the verdict is `confirmed`; otherwise `switch` (a top-level navigation through YouTube's own sign-in address, `next=` the tokened watch URL; bounded back-off `backoffMs` before the 2nd attempt) and, after `MAX_SWITCH_TRIES` = 2 (menu already says yes but the page disagrees) or 1 (menu says no), `fail` → `switch-failed`, never clicked. A hold point (Pause) precedes every switch.

Gates (a click needs a fresh `confirmed` each time): (a) after the switch (verified step), (b) `recheck` immediately before the click: re-reads the page session, the menu and the header picture, (c) the verify page after the click: the page session must still be the target, otherwise `identity-unclear` (what the page shows is not that channel's; nothing is recorded). The like state is only read after (a); `already` / `liked` entries are only recorded for items that passed these gates.

Order planning: when the page is already on another ticked channel of the container (same comment, nothing done yet), the two queued items trade their channel (`chid, chname, chhandle, ck`): that channel is done first and one switch is saved (`orderFirst`).

Edge cases: personal vs brand (main has no delegated id); selected channel missing from the menu (renamed / deleted / needs re-sign-in) → skipped with a clear reason and the channel is flagged in the channel list (`p9bn.chstale.<container>`, cleared by a refresh); account with one channel; duplicate channel names (ids, not names); signed-out container (`not-signed-in`); slow ytcfg (waits 6 s); identity drift mid-run (verify gate). Not detectable in the page: consent / "choose an account" interstitials live on other hosts (`consent.youtube.com`, `accounts.google.com`): the content script is not injected there, so the item ends as interrupted by the staleness rule (`pageKind` is ready for it should a script ever run there).

Limits: real YouTube was not available; the ytcfg/masthead formats are the ones YouTube is known to use (yt-dlp / YouTube.js) and mocked in `tests/mock-youtube.js` (knobs `ghostSwitch`, `identNever`, `mastAvatar`, `ytcfgDelay`, `driftAfterClick`).

Tests: `tests/unit11.js` (module), `tests/e2e-safety.js` R30 (never reachable), R31 (follows at the 2nd attempt), R34 (Oluchi's 3-channel case), R35 (matrix: 1–4 channels × any ticked subset × page session on any channel × lag / unreachable), R36 (conflicting header picture, slow ytcfg, drift after the click, missing channel, duplicate names).

16. 1.1.6 - live capacity and the hardening audit

16.1 /capacity

- **What the server counts** (`bean_api.capacity_of`): `max_likes = channels_saved - channels_used - reserved_by_queued_jobs`. `channels_saved` is the list Bean pushes with `PUT /agent/config {channels: savedKeys()}`; `channels_used` comes from the server copy of the ledger.
- `savedKeys()` (ext11.js) = ticked containers x ticked channels that are still in the channel list and **not flagged**. A channel is flagged when a run writes `p9bn.chstale.<cid> = {at, ids[], why}`: `not in the account menu` (renamed / deleted / needs sign-in) or `the channel could not be switched to` (switch-failed). The flag is lifted by a successful channel refresh (`runDiscover`), by a like / confirmed "already" from that channel (`clearStale` in `recordFinal`), or

after 24 h. The popup's "Possible likes" uses the same filter (`healthyTargets`), so Bean and the API show the same number.

- **Freshness:** `pushConfig` runs one request at a time; a change made while a push is running is pushed right after; a failed push is retried after 2 / 4 / 8 ... 60 s. The runner reloads containers and channel lists when settings change (a stale container list had made it drop a new selection) and when a `chstale` flag changes. `p9bn.apistat.*.cfg` ({keys, at, fail, rm}) says what was last pushed (diagnostics).
- **Cleared marks:** every ledger removal (`remove` , `remove-already` , `remove-comment` , `clear`) queues rows `[vid, lc, channelKey|"*"]` in `p9bn.ledrm` ; `mirror()` sends them with `POST /agent/ledger/remove` (before pushing new entries), keeps them while the server is unreachable, and drops them on 404 (older server: its copy stays, said in the UI note). "Restore from server" flushes the queue first so cleared marks cannot come back. A full `pushAll` only re-sends what the local ledger still has.

16.2 Hardening audit (what was checked, what changed)

Area	Finding	1.1.6
Page-load waits	the channel read waited for `readyState === 'complete'` (images, comments, ads) before even asking the switcher (a fetch)	waits for the markup (`!== 'loading'`); the VPN switch still waits for `complete`
Comment locator	already adaptive (35 s, up to 60 s while loading, nudges, reply expander, found by lc id, not by position)	unchanged
Like-button read	reads `aria-pressed` of the button inside the located comment; language independent	unchanged
YouTube Focus	gate waited 6 s + 4.5 s + 4 s; YTF is slow right after a switch	10 s + 7 s + 7 s; absent / off is still a deliberate stop (`no-ytf` / `ytf-off`); the comment is found by id regardless
Pause / stop	pause lost to stale writes; no hold on the switch path	own key, hold points at item start, before a switch, before VPN / like
Storage hand-over races	`patch` / `mutate` are read-modify-write on `p9bn.batch`; two pages can overwrite each other's field	pause is single-writer; other fields are idempotent and re-derived; a lock / per-field merge is a candidate for 1.2
Tab lifecycle	crash / close / navigate-away end as `interrupted` through the staleness clock	unchanged; consent / account-chooser pages are on other hosts (no content script) and end the same way
VPN timing	the answer is adopted by the next page, never switched twice	unchanged
API jobs	jobs for flagged channels are no longer promised by `/capacity`	capacity only; the job path itself was not changed
Run log	identity evidence per item in plain words	added in 1.1.6
Player independence	`ytd-player #error-screen` was read as an "unavailable" hint; with YouTube Focus 1.4.0 blocking the player scripts (comments-only) it would mean nothing	removed: Bean reads only the comment DOM, the page's own playability message and `data-ytf-*` attributes; never `movie_player`, `<video>` or googlevideo

17. 1.1.7 - PLAN (written before the code; status table at the end of the section is updated as work lands)

Goal: Bean runs for hours without anybody watching, like YT Panel: one **persistent queue of orders** (popup orders and API jobs alike), an **Orders section** to see/steer it, **per-link channel reservation**, **long-run stability**, a **typed error catalogue** with safe retries,

and tests that prove it (accelerated multi-hour soak + error paths). Built in `/workspace/panel9-bean-117` from the 1.1.6 tree; 1.1.6 stays untouched (published). Rules that do not change: at most ONE Bean tab at a time, a like is never clicked twice (ledger + "like sent" markers + verify-only mode), identity gates of 1.1.6 stay, never touch `movie_player` / `<video>`.

17.1 Concepts

- **Order** = one request to like a list of comment links: from the popup ("Like comments" button) or from the API (a job the server handed to this install). The unit the queue holds. The run record `p9bn.batch` stays what it is (the live state of the ONE order that is running); the order is the durable envelope around it.
- **Queue** = ordered list of orders in `p9bn.orders` (FIFO by arrival; the popup can move / cancel not-started orders). **One order runs at a time** (still one Bean tab at a time). The old "A run is already in progress - stop it first" refusal and the API's `busy` error (waited 90 s) are gone: new work waits its turn.
- **Reservation** = for every queued/running order, per link (`vid|lc`) the set of channel keys (`ck` : `p:<id>` / `h:@handle` / `c:<container>`) that order will use. A new order for the same link only sees channels that the ledger has NOT used and no other open order has reserved: it is planned without visiting those channels (nothing is opened to find out). Released automatically when an order ends: channels that did not like (failed, cancelled, interrupted) become free again; channels that liked are in the ledger.
- **Executor** = the one page that dispatches (hidden runner when its heartbeat is fresh, else the popup). It holds a **lease** key `p9bn.ordlease` (renewed every tick; taken only when absent/stale >10 s) and is the ONLY writer of `p9bn.orders` (Panel 9 merges storage per key across sessions by whole snapshot: one writer per key). The popup talks to it through the append-only request key `p9bn.ordin` (submit / cancel / move / retry / clear-history) and reads results from `p9bn.ordack` ; when the popup is itself the executor it applies requests in the same tick.

17.1b Quantity per link (added 18:13 at Oluchi's request; popup and API share it)

- Popup: a **Likes per link** field (empty = all possible) and, with several links, a per-link field; each shows `max N` = ticked channels minus used (ledger) minus reserved (open orders), live. A value above max is clamped (and said). The button/plan line reads `Like 5 of 12 possible` .
- One logic for popup and API: an order link has `quantity` (`null` = every available channel). The order reserves exactly `quantity` channels per link, chosen by a cost model (`pickChannels` : a container's active channel costs no switch; extra channels in a container that is visited anyway cost 1; opening another container costs 3; so containers are filled before another is opened, ties go to the container with more channels left and to the spread). When likes fail, the order continues with **spare** channels (not used, not reserved by another order, not already tried by this order, not in a container that was signed out / bypassed) until `quantity` is reached; a channel that failed is never clicked again (click-class errors are `retry: none` ; safe-class ones get at most 2 retries on the same channel BEFORE a spare is used). The order is `done` when every quantity is met, `partial` / `failed` with the typed `shortfall` error ("2 of 5 requested likes were made: no spare channel was left for the other 3") otherwise. 'already liked' counts as a like, as in the API.

17.2 Storage keys (single writer each)

key	writer	content
`p9bn.orders`	executor (lease holder)	`{v, seq, orders:[order…]}`; open orders first, then history (cap 60, results trimmed)
`p9bn.ordin`	popup	`[{id, op, …, at}]` requests, pruned by the popup once acked
`p9bn.ordack`	executor	`{ids:[last 300 handled request ids], res:{id: {ok,msg,oid}}}`
`p9bn.ordlease`	executor	`{role, id, at}`
`p9bn.ui`	popup	`{ordRun, ordQ, ordH}` collapse state of the three panels (remembered)
`p9bn.hk`	executor	last housekeeping: sizes, pruned counts, vpn health, tab recycles, stalls

Order fields: `id, seq, src('popup'	'api'), jobId, created, state, links[{n,vid,lc,quantity	null,url}], targets (snapshot of ticked containers/channels), pace, restoreBack, res{'vid
States: `queued -> running -> done	partial	failed

17.3 Dispatcher (lib/orders.js, pure; glue in ext17.js)

`Dispatcher({store, clock, ports})` is a deterministic state machine (so the soak test can drive it with a fake clock and a fake "tab"). One `tick()` every second on the executor:

1. renew/take the lease; read requests (`ordin`) not in `ordack` ; apply (submit = plan + reserve + enqueue, cancel, move, retry, clear-history); write `ordack` .
2. reconcile the running order with the run record: batch of that order over -> finalize (counts, rows, error, release reservation, history); batch missing / stale beyond `RESUME_MS` after a crash or Panel 9 restart -> `interrupted` : the order goes back to the head of the queue with `attempt+1` (its liked items are in the ledger, so the re-plan can only contain what is still open; items whose like was SENT stay verify-only through their markers). After 3 interruptions it ends `failed` (`tab-lost`).
3. nothing running and the head is startable -> pre-checks (Focus, VPN, containers, channels) with the typed catalogue; start = re-plan from `res` (assigned channels first, API orders top up from free channels) -> `batch` record -> `advance()` opens the first container. API orders are reported to the server (heartbeat every <=20 s while queued-locally AND running, results as they come); a server-side cancel cancels the order.
4. safe retry (17.5) of failed items after a batch ends, as a new batch of the same order (max 2 retries, only for retry-safe error types and only when no like was ever sent for that item).
5. housekeeping every 10 min (17.4).

17.4 Long-run stability

- **Caps**: item step log `lg` <= 40 lines, run `events` <= 20, `p9bn.runs` <= 5 runs and <= 900 KB, order history <= 60 with `rows` <= 400 each, `p9bn.tabidx` <= 60, `p9bn.ledrm` <= 20000, `p9bn.clk.*` markers pruned at 60 days (hourly), `p9bn.chstale.*` at 24 h, `p9bn.diag` <= 30; ledger keeps its own cap (100k pairs). `housekeeping()` reads `getBytesInUse` and, above 6 MB (quota 10 MB), trims history/runs/markers in that order. All numbers are exported constants.
- **Tab recycle**: a tab that has handled 40 items or lived 15 minutes (Panel 9 2.3.23 expires a background tab after 20) finishes its current item, marks its group `queued` again with `recycle:n` and goes idle; the popup/runner closes it and opens a fresh tab for the same container (new VPN location first when rotation is on). Same record, same order; no like is repeated (finished items stay finished).
- **Stall watchdog**: a group whose item has been `running` for >4 min with no state change (heartbeat can be fresh while the page is stuck in a spinner) is handled by the executor: item without `sent` / `verify` -> back to `queued` (max 2 stalls per item, then `failed: stalled`); item with `sent` / `verify` -> `liked-unverified` (counted, ledgered, never clicked again); the tab is closed and recycled. Never between click and verify (those items are never re-queued).
- **VPN health**: before an order starts, every 10 minutes while orders run (`checkNow` , state-only, touches nothing), and after any `vpn-failed` : result shown in Orders; two consecutive failed checks while an order runs pause the order (`vpn-down` , waits like `waiting`), resumes on its own when healthy again (max 30 min, then fails typed).
- **Capacity refresh**: config push (`/agent/config`) after every order, every 2 min while API is on (existing), and a forced push every 10 min when idle; stale channel flags are lifted at 24 h. Bean does NOT auto-open tabs to re-read channel lists unattended (a deliberate choice: unattended tab opens stay limited to orders).

17.5 Error catalogue (lib/errors.js)

Every code that can end an item or an order has one entry: `{code, type, retry, wait, message (plain English), hint}` . `type` in `focus | vpn | identity | page | click | account | capacity | system | user` ; `retry` in `none` (never: a click may have been sent, or it can never succeed) | `safe` (nothing was clicked: retry is allowed, <=2 times) | `wait` (retry after a pause, whole queue). Invariant tested: no code that can occur after a click has `retry != none` ; every code of `links.js CODES` has an entry; messages never contain internal ids or stack text. The popup (History) and the API (`results[].message` , `error.message`) use the same sentences.

17.6 Run-record race

The run record is read-modify-write from 3+ pages. 1.1.7: (a) every writer serialises its own writes (already so in content.js, added in popup.js `mutate`); (b) writes carry `rev` and are **verified**: after ~250 ms the writer re-reads; if the changed fields are not there (a stale mirror write won) it re-applies its idempotent mutation (<=3 times). Combined with the existing fin-memory (finished items never revert), a lost update is repaired within a second instead of showing up as a wrong state. `mergeFields()` (pure) is unit-tested with injected losses.

17.7 Orders section (popup)

Three collapsible panels in the Panel 9 style (no cards/outlines; `popup.css` tokens; remembered in `p9bn.ui`): **Running** (order, source, progress, current container/channel, ETA, VPN health, Pause/Stop shortcuts), **Queue** (not-started orders with position, links, reserved channels per link, up/down, cancel; "waiting" reason), **History** (last 60: source = Popup / API job id, links, liked/already/failed, plain-English reason, copy). The old results list under the buttons stays as the live view of the running order. "Like comments" with a run in progress now says "Added to the queue (position N)".

17.8 Tests

- `tests/unit17.js` (node): order model, reservation (two orders same link, release on cancel/failure, API top-up), move/cancel rules, interruption recovery, error catalogue invariants, race merge with injected lost writes, caps.
- `tests/soak17.js` (node, fake clock, **accelerated 12 simulated hours**): the real Dispatcher + housekeeping + stall/recycle logic against a simulated tab pool with random outcomes and injected faults (tab crash, stall, VPN down, popup closed, Panel 9 restart, duplicate request, storage losses). Asserts: no (video, comment, channel) liked twice, queue drains, no order stuck, reservations released, storage bytes bounded, at most one tab, every error typed.
- `tests/e2e17.js` (Electron, real Panel 9 + real YouTube Focus + mock YouTube + real bean_api): queue while running, reorder/cancel, popup closed mid-queue (runner carries on), API job joins the queue behind a popup order, per-link reservation without visiting channels, safe retry after a page error, collapse state remembered. Regressions: `e2e11.js`, `e2e-safety.js`, unit suites; section-21 'unknown-state' flake isolated into its own re-runnable case (`e2e.js` section 21 under `BEAN_ONLY=21`).

17.9 Limits and open points (said plainly)

- Panel 9 starts the hidden runner only while the API is on: with the API off and the popup closed nothing can run the queue; it waits (persisted) and resumes when the popup is opened.
- The server hands this install ONE API job at a time, so API jobs behind the first stay "queued" on the server (they cannot be listed or reordered from Bean without a server change). The first one joins the local queue as soon as it is handed out.
- Real YouTube is not available to the build: soak and error paths use the mock; behaviours that depend on YouTube's live pages stay as verified in 1.1.6.
- A dispatcher retry round creates NEW items (new batch), which rotate the VPN location again; only a re-opened or handed-back item keeps its location.
- The Orders history shows a source tag per row (popup / API) in the Running / Queue / History panels (the plan called the three panels Running order / Queue / Sources).
- Background tabs are off by default (1.1.6.5, section 22: `p9bn.bgTabs === true` only; Panel 9 2.3.23 hides such a view and YouTube comments never load in it). 1.1.7 adds the settings switch (17.12). Not pulling focus needs a fixed Panel 9 (2.3.24+). The reload-survival logic (17.11) is Bean-only.

17.11 A tab that disappears mid-step (added after Panel 9 2.3.23: a background tab expires after 20 minutes)

A working tab touches the run record at least every ~15 s and answers the `p9bn.ping` / `p9bn.alive` probe at any time. `advance()` (popup or runner) probes when the record has been quiet for 35 s, no pause is requested and no navigation grace is open; no answer within 4.5 s = the tab is gone (or dead). Then `O.tabGone` (lib/orders.js, pure, unit-tested) runs inside `mutate`: the group goes back to the queue with `recycle+1` (a fresh tab opens next, normal pace/VPN rules), items that were `running` go back to `queued` (`tabGone+1`, log line), **except** items with `sent` / `verify`, which become `liked` / `liked-unverified` ("Like sent - not confirmed") and are never queued again. The page's own like marker (`p9bn.clk.*`) stops a re-click that the record does not know about (the fresh tab ends such an item `click-sent-before` or confirms it). After `MAX_TABLOST` = 3 losses of one container its open items end `tab-lost`. The tab recycle limit was lowered from 25 to 15 minutes so Bean replaces a tab before Panel 9 expires it. Tests: unit17 (tabGone), e2e R69a (tab closed before any click) and R69b (tab closed right after a like was sent).

17.12 Settings switch "Background tabs (needs Panel 9 2.3.24+)"

A plain checkbox (`#bgTabs` , the same `label.opt` look as the other switches) under the pace row of the popup. Default off. Ticking it writes `chrome.storage.local['p9bn.bgTabs'] = true` ; unticking removes the key (never `false`); it follows the stored value (onChanged). The key is the only state: `openInContainer` and the discovery opener read it fresh before every tab open (`bgTabsNow()`), so it applies at once to every run - popup orders, API jobs and queued / YT Panel orders (they all go through the same executor page) - and each run records `openMode` and one `open mode - ...` event (ext17 `startBatch` , popup `goDirect` , legacy ext11 job). Tests: unit11 (strict `bgTabsOn` , call sites, checkbox + wiring), e2e R68 (default / true / `'true'` / false / 1, run log, the switch writes and removes the key), R63 (no focus with the setting on).

17.13 A container's tab goes silent (from Oluchi's real job job_7JCK, 3 of 9 containers ended "container tab stopped answering")

Before: when the run record had not been touched for `staleAt` (updated + 45 s, or the open grace 75 s / navigation grace 120 s), `advance()` ended every open item of that container "not-started" - no second try. *Now* (`lib/orders.js reopenGroup` , called from `popup.js advance()`):

- **Reopen before fail.** The silent tab is closed (verified close, see 17.15), the group goes back to the queue (`recycle` +1, `reopens` +1) and the container is opened again in a **fresh tab**. At most `C.MAX_REOPENS = 2` reopens; after that the items end with a plain message. Items that were open go back to `queued` ; **an item whose like was sent (or was being verified) is counted `liked-unverified` and never clicked again**; the item keeps its `vpn.location` (`vpnBeforeOpen` does not rotate again).
- **"Never answered" vs "went quiet".** The content script writes `p9bn.tabseen.<run>.<container> = {url, at}` at `document_start` (the URL without Bean's tokens). No marker = no Bean page ever ran in the tab (Panel 9's error page, a dead connection, a Google page): *"Its container's tab never answered ..."*. A marker = *"... went quiet after it started ..."*. The marker is removed with the run (`cleanMarkers` ; leftovers older than 6 h are pruned at start and by housekeeping).
- **Google pages.** `google.js` runs only on `accounts.google.com/*` and `consent.youtube.com/*` (manifest content script + host permissions). It never clicks or types. It finds Bean's run in the page address (`continue=` / `next=` keep it) or, without it, only when exactly one container is "opening", no Bean page has started in it and it was opened < 150 s ago. It writes `p9bn.gcheck.<run>.<container>` ; the runner skips that container at once with code `google-check` : *"Google asked container X to sign in or verify; open it once and complete it."* Nothing is solved.
- **Open grace.** The grace now starts when the tab **opens** (it used to start before the VPN rotation and was wiped by the VPN heartbeat): 75 s, or 120 s when a VPN rotation finished in the last 30 s (`openGraceMs`).
- **Heartbeat-missed log.** On expiry / tab-gone the run log gets one line: VPN state and location, whether Bean's own page still has a runtime id, open/VPN-ready/last-heartbeat times, the tab-seen marker (or "no Bean page ever started"), the last ping answer (host, script runtime id). Panel 9 gives Bean no tab URL, so none is logged.

17.14 Split writers for the run record (job_7JCK: 83 writes of `p9bn.batch` in ~70 s, 14 distinct versions, older ones rewritten after newer)

Panel 9 mirrors `chrome.storage` between sessions by snapshot copy; two pages doing whole-record read-modify-write on one key lose fields (the runner's VPN location, grace, tab id) and bring old versions back, and the watchdog then expired at exactly 75 or 120 s.

- **One writer for the record.** `p9bn.batch` is written only by the page that owns the run (popup or hidden runner = "the runner"). A YouTube tab **never** writes it (`content.js patchNow` no longer calls `sset(K.batch)` ; unit-tested).
- **Tabs write their own keys.** `p9bn.rep.<run>.<container>.<slot>` = the tab's *cumulative* report (item fields, group fields, appended events; whole key, only this tab writes it - a lost or replayed write costs nothing) and `p9bn.hb.<run>.<container>.<slot>` = a tiny heartbeat `{gen, seq, rseq, graceMs, idx, at}` . `slot` 0-11 is the first free slot (a replaced tab keeps its own until the runner has merged it); `gen` = the tab's start time. The tab's own view of the run is the record + its own report (`readB` , `overlay`).
- **The runner merges** (`popup.js pullReports` , every pass, before it decides anything): `lib/reports.js apply` takes only what is **new** in a report since the one applied last (`diffRep`) - so a deliberate change (a resume, a requeue) is never undone by an old value still in a cumulative report; a finished item is never touched, a finished group stays finished, the run state only moves running -> done/stopped, a run that is over takes nothing. Reports of a replaced tab (`gen` older than `repCut` , set when a tab is opened) are dropped.
- **Watchdog from the runner's memory.** `mem[run:container]` = open time, VPN-ready time, tab id, last heartbeat **seen** (own clock), grace. A group is silent when `now > max(lastSeen + 45 s, grace, pause)` ; the age of the shared record is

not used any more (a stale copy resets it). The 35 s "tab gone" probe, the tab sweep (`activeKeep`) and the expiry all use it. A tab that still answers the alive ping gets one more period before it is replaced.

- **Re-assert.** If a stale copy loses a group's tab id or open time, the runner puts them back from its memory (`tabOf` , `mem`) on the next pass; the old `rev` / `verifyWrite` repair of the runner's own writes stays.
- **Stop signals.** "The run is over" is only believed when definitive: the run's stop key `p9bn.stop.<run>` (written by Stop / Clear) or **its own record** (same run id) not running on 3 reads in a row. A missing record or another run's record (a stale copy of the previous run) is ambiguous: nothing is skipped; the container goes back to the queue (`handBack`). (Real case run 7ln17k8fmvzclqr2: item 1 skipped 2 s after the start - "failed (stopped) before the tab opened".)

17.15 VPN first, one tab at a time

- **Never open or navigate while the VPN is down.** For runs that use the VPN, the runner waits (`B11.vpnUp` , a read-only state poll, up to 5 min before a tab opens) until ExpressVPN reports `connected` and a ~3 s settle (counted from when it was *seen* connected after a non-connected state, or from the end of Bean's own rotation). A tab asks the runner for the same wait (`p9bn.vpnreq` with `up:true` , up to 150 s) in `goTo()` before every navigation (next item, channel switch, fresh reload, verify). The rotation itself only ever starts while no Bean tab exists (before a container's first tab) or while the tab sits idle in the pace gap and navigates only after the answer. If the VPN never returns, the open fails the item `vpn-failed` (nothing loaded); an in-tab navigation proceeds after 150 s (logged) and the popup recovers a tab that then never starts.
- **Tab registry per tab.** `p9bn.tab.<0..39> = {id, run, cid, at}` (written by the opener, removed by whoever closes) replaces the single `p9bn.tabidx` list that two pages rewrote whole (real job: tabs left open in containers 25, 26, 27, 30, 31). The sweep (`sweepTabs` / `sweepVerified`) reads the 40 slots, so a stale copy can only lose one entry, not the list. The next tab is never opened while a registered one will not close (one-tab rule, 8 tries).

17.16 The API server's jobs in the Queue panel (read-only)

- The popup's **Queue** panel lists, under a quiet "API server" heading, what the server holds for THIS install: the running job first, then the waiting ones in line order. Each row: place (`running now` / `#2 in line`), status (Running / Waiting / Stopping), the likes asked, when it was created (**WAT**, fixed UTC+1), the caller's `note` (e.g. "YT Panel batch 7", "YT Panel retry 7.1"), and its links as plain text (`youtu.be/<video> · comment <id> · N likes` ; the link is rebuilt from the 11-character video id and the comment id, never taken as an address from the server). A job Bean already took into its own queue says "Bean has it".
- **Server:** `GET /api/bean/agent/jobs` (install-token auth, scoped by install id, read-only, max 60 jobs, status `queued` / `running` only; no key, token or hash in the answer). Proposed diff: `server-patch-agent-jobs.diff` (applied only to the test copy `tests/server-patched/bean_api.py` ; **not deployed**). An older server answers 404 and the popup says "This server cannot list its jobs yet." and keeps working.
- **Client:** `BeanApi.jobs()` (lib/apiclient.js), `B11.apiJobs()` (ext11.js), pure view logic in `lib/apijobs.js` (unit-tested), drawing in `ui17.js` (text only, no markup). Refreshed every 10 s while the popup is open and visible (one small GET per refresh; the server's agent rate limit is 240/min); a failed refresh keeps the last list and says so; API off = no block at all.

17.17 Late 1.1.7 fixes found by the final matrix

- **VPN drop while a fresh tab has not answered** (popup.js, before the quiet-tab check): if ExpressVPN leaves `connected` after a tab was opened and that tab has not sent a single heartbeat, the tab is closed and the container reopened as soon as the VPN is connected again plus the 3 s settle - no 75-120 s open grace and no 35 s quiet period. Bounded (`g.vpnReopens` <= 6) and not counted as one of the 2 tab reopens.
- **A report never hides what a tab already saw**: `readB()` in content.js overlays the reports of the EARLIER pages of the tab even before the page has a report of its own (the first reads of a page after a channel switch decide which item it takes; R27/R28/R35).
- **The record's pause copy**: after a stale write wipes `pause` , the runner restores it from what the tab last reported (`pausedBy`), as it restores `pauseReq` from the Pause key (R32).
- **Runner log lines survive a report merge**: `R.apply` unions the item log (`lg`) by time instead of replacing it with the tab's copy.
- **A frozen runner does not end the run**: the dispatcher's lost-run backstop gives the tabs 2 minutes after THIS page was itself frozen (> 30 s between two checks: sleep, a starved window); a run whose items were all final when the backstop (or the resume window) ends it is `done` , not `stopped` .
- **Mock test hooks**: `E11_FAST` , `E11_STOP_AFTER_STALE` (e2e11), `R35ONLY` (e2e-safety) shorten debugging runs; they change nothing in the product.

17.10 Status (updated while building)

item	status
plan (this section)	written
queue, Orders panels, reservation, quantity/spares/shortfall, caps, recycle, watchdog, typed errors, run-record race	built, tested (unit17, soak17, e2e17 R50-R55, e2e11, e2e-safety)
1.1.6.4 port: background flag, reload survival (also for the queue), R63-R67	built, tested
tab disappears mid-step (17.11)	built, tested (unit17, R69a/b)
1.1.6.5 port: background tabs off by default + open-mode log + settings switch (17.12)	built, tested (unit11, R68)
API server jobs in the Queue panel (17.16)	built, unit-tested (unit17, server_test); server endpoint NOT deployed
final counts	see the build report / .build-notes.txt

18. 1.1.6.1 — channel-switch hotfix (from the real 1.1.6 run 1yp0desbmvy38sqj)

Observed (real 1.1.6, 3 containers): item 1 (page already on the channel) is confirmed and liked. For every item that needs a switch, the sign-in address is followed, the page loads twice (attempt 1, attempt 2, ~16 s apart), the account menu (`/getAccountSwitcherEndpoint` , `no-store`) lists the TARGET as active, but the page's own ytcfg session (`DELEGATED_SESSION_ID` / `DATASYNC_ID`) and the header picture are still the OLD channel on both pages → `switch-failed` . The 1.1.6 gate did its job (no like was ever sent as the wrong channel); the switch itself never completed.

What is certain / what is not. The two signals come from the same server at nearly the same moment, so either (H1) the document the tab holds is a stale COPY of the page (HTTP cache / service worker / history entry answering a navigation whose address is identical to the one the tab showed one second earlier — 1.1.6 used the SAME tokened address as `next=` — and again identical on attempt 2), or (H2) the server really renders the old identity for that load and the sign-in address did not change it. A box without a signed-in YouTube cannot tell the two apart, and no public documentation says which one it is. So 1.1.6.1 does not guess: it MEASURES it on every mismatch and handles both.

Facts established by the research (public sources, YouTube Focus / Panel 9 source):

- YouTube's own channel row uses `/signin?action_handle_signin=true&authuser=<Google account index>&pageid=<brand channel id = DELEGATED_SESSION_ID>&next=<url>&feature=masthead_switcher&skip_identity_prompt=true` ; `authuser` selects the Google account, `pageid` the channel. Bean already used the item's own `accountSigninToken.signinUrl` (complete with both) — the address was not wrong.
- `ytcfg` derives `X-Goog-AuthUser` from `SESSION_INDEX` and `X-Goog-PageId` from `DELEGATED_SESSION_ID` / `DATASYNC_ID` (`<pageId>||<userId>`); like requests carry those, which is why the page's ytcfg (not the menu) is the signal that decides a click.
- YouTube watch pages are sent `Cache-Control: no-cache, no-store, max-age=0, must-revalidate` , so a plain HTTP-cache hit is unlikely; a service worker's cache storage, a bfcache/history snapshot, or a speculative prefetch is not excluded by those headers.
- Panel 9 2.3.21 `ytf-lean` never cancels `/signin` , `/getAccountSwitcherEndpoint` , accounts.google.com or main-frame navigations, and wipes YouTube's service-worker registration / cache storage only once, when the "Comments only" switch turns ON — so it is not the direct cause, but it does not keep a re-registered worker out later.

The fix (identity gate unchanged: page ytcfg + account menu + header picture must agree before ANY click):

1. `next=` of the sign-in address is now a UNIQUE address (`p9bt=<time>`), so the page that comes back can never be answered from the copy of the page shown before the switch.
2. On a post-load mismatch (menu says target, page does not) Bean now logs the document's delivery facts (navigation type, redirects, worker/cache) and fetches a FRESH copy of the same page (`fetch` , `cache:'no-store'` , same origin, unique address; read-only) and reads its ytcfg:
   - fresh copy says **target** → the tab holds a stale copy: remove YouTube's service worker + cache storage, load the page again from a unique address (`refresh` , no sign-in, max `MAX_REFRESH` =2), confirm again;

- fresh copy says **another channel** (or cannot be read) → the sign-in did not take: switch attempt 2 first tries the PAGE'S OWN account menu (avatar → "Switch account" → the channel's row, a plain click on menu rows, never anything near a like; locale-independent row match for the common languages, channel row matched by name) and falls back to the address if the menu cannot be used or does not navigate within 15 s.

3. Bounds: `MAX_SWITCH_TRIES` =2 and `MAX_REFRESH` =2 (≤ 4 page loads per item); then `switch-failed` as before, nothing clicked. The run log of every item carries the evidence lines (so the next real run settles H1 vs H2 from the log alone).
4. Tests: mock knobs `staleDoc` (the first N navigations after a sign-in are answered with the old copy, a `fetch` gets the fresh page) and `uiOnly` (the sign-in address changes the menu but never the page identity; only the page's own menu click does; `uiMenu:false` removes the menu). e2e-safety R39–R42; 1.1.6 FAILS R39/R41 exactly like the real run (menu = target, page old, 2 sign-ins, switch-failed).

Not proven: that real YouTube behaves like either mock model. If neither the fresh copy nor the page menu settles it, the run still ends in `switch-failed` with the evidence lines in the log — never a wrong like.

19. 1.1.6.2 — second channel-switch hotfix (from the real 1.1.6.1 run n3bkmbqymvy96cfy)

Observed (real 1.1.6.1, 2 switch items): the document came from the network (0 redirects, no service worker, ~250 kB), the page identity (ytcfg, header picture) was the OLD channel, but an in-page `fetch()` of the same URL (no-store, unique address) rendered the TARGET; two further same-tab reloads (`refresh 1/2` , `2/2`) still showed the old channel; the page's own menu fallback found no "Switch account" row (text matching, interface language); containers that failed to switch in the run before opened ON the target in this run. H1 (stale copy in the tab) is disproven.

Panel 9 audit (2.3.20 source on the box; Helper2: 2.3.21 d5ff08a): no difference between a top-level navigation and a fetch: the header rewriter (`onBeforeSendHeaders`) changes only UA and Sec-CH-UA for every request type of a tab; no cookie or authuser injection (`withAuthUser` is only used by Panel 9's own account switcher), no protocol intercept for youtube.com, one cookie jar per container session. Bean's tokened watch address carries only `p9bean` , `p9bnc` , `p9bt` (nothing identity-related). So the difference is on YouTube's side (a document request is answered differently from a fetch, and/or the sign-in's cookies finish late). Not provable without a signed-in YouTube account; hence Bean measures and uses the path the evidence supports.

Fix:

1. `lib/identity.js nextStep(..., ro)` : fresh copy = target and the tab disagrees → `reopen` (≤ `MAX_REOPEN` =2) before `refresh` (≤ 2) before another sign-in. A fresh copy that cannot be read at attempt ≥ 2 → one `reopen` .
2. `content.js reopen` step: no click, no navigation; the item goes back to `queued` (`ro` +1), the group to `queued` with `recycle` +1 and a 6 s gap; the popup's `advance` closes the tab (never two tabs) and opens a NEW tab for the container (the open key carries `recycle` , so the container may be opened again). The VPN rotation happens as for any container open. The identity gate runs again in the new tab: if it shows the target the like proceeds, else the same bounded path continues.
3. Settle poll: before the diagnosis is used, up to `SETTLE_POLLS` =3 more fresh fetches, 5 s apart, until the server renders the target (the `/signin` interstitial may still be setting cookies); nothing navigates meanwhile.
4. `uiSwitch` is language-independent: `switchRows()` takes the rows of the menu's FIRST section that have no `<a href>` (never the last of ≥3 rows = Sign out) and requires exactly ONE candidate; the click is verified by the channel list opening; the channel row is chosen by picture (normalised avatar) then name. No label text is read (static test: no non-ASCII, no label regex).
5. Bounds per item: 2 reopens + 2 refreshes + 2 switch attempts (each page load bounded as before); then `switch-failed` , nothing clicked.
6. Mock models: `navLag` (navigations inside a tab render the previous identity until a tab is opened anew — `Sec-Fetch-Site: none` — while fetches render the target), `settleMs` , and menus in en, ru, ar (RTL), ja, de, pt, fr, hi, sw and an unlabeled one (`menuLocale` , `menuNoSwitcher` , Sign out recorded at `/__signout`). Tests R43–R47.

Not proven: that real YouTube/Panel 9 behave like the mock models, and that a new tab shows the target in every case (it did in the one real observation). A real signed-in test account on the box would be needed to prove it; the run log of the next real run decides.

20. 1.1.6.3 — hotfix from the real 1.1.6.2 run z09pnf5pmvyrefxt

- **Verified close (popup.js).** `closeTabId` sets `closedTabs` only after `bridge().closeTab` succeeded (or said "not a tab opened"), retries 3x in place, and shares the call in flight (`closing` map); before 1.1.6.3 a concurrent caller was told "closed" while the call could still fail and the registry entry was pruned, and after 6 failures the tab was forgotten. `regTab` reads the registry back (3x). `sweepVerified(keep)` sweeps, re-reads the registry and reports `{ok,left}` ; `advance` calls it before every open and does not open while `left` is non-empty (`tabCloseFails` , event line, 8 tries then `open-failed`). `stop()` and the end of a run use it too, and `stop()` sweeps again after 5 s and 15 s. Test hook: `self.__beanCloseFail = N` makes the next N close calls fail (mock only).
- **Verify never says not-found (content.js).** In verify mode (the safety marker exists) every early exit that is about the page, not the like (comment missing, no button, comments off, playability message, identity not confirmed, state unreadable) returns `verify-retry` ; the tab reloads from a fresh address (`vtries` , 1.5 s·n back-off, `VERIFY_TRIES = 3`) and finally records `liked-unverified` (status `liked` , kept in the ledger) with the reason. No code path after the marker can click.
- **Click line.** `clickEv` is written into the marker with the click; a verify page without a `click` step restores it from there. `onSent` is awaited (2.5 s bound).
- **VPN.** `vpnBeforeOpen` keeps `item.vpn.location` when present; rotation happens once per like attempt (a reopen or hand-back of the same item is not a new attempt; a retry round creates new items, which do rotate).
- **Switch speed.** A page that follows a switch (tries>=1, ro<1) whose ytcfg session is still another channel (`idPre.verdict === 'mismatch'`) calls `reopenNow` BEFORE the switcher read; `straight` (tries>=1, ro<1, page not confirmed after the read) skips the fresh-copy probe and the settle polls and reopens at once; MAX_REFRESH=0; reopen gap 1.5 s; SETTLE_MS 3 s.
- **Load.** Eco pace; report timer 15 s and only for visible pages; API loop idle sleep 8 s; vreq fallback poll 4 s; `renderApi` only when visible.
- Tests: unit11 (54), e2e-safety R60 (tab leak, two cases), R61 (verify retry: off x2, off x3, empty x1), e2e11 section 2 (one rotation per like through a fresh-tab reopen).

21. 1.1.6.4 (carried into 1.1.7) — no focus stealing; runs survive an update

- **Focus audit.** Bean (extension) has no `chrome.tabs` / `chrome.windows` calls; the only `.focus()` is the popup's own text box (popup page only, never the runner); `scrollIntoView` scrolls inside the page and never moves OS focus. The focus/raise came from Panel 9: `openTab` -> `createTab` (activate default true) -> `activateTab` -> `webContents.focus()` + `ctx.activeId` = the new tab. Bean now sends `background: true` with `openTabInContainer` ; `panel9-bean-nofocus.patch` makes Panel 9 create such a tab with `activate:false, noFocus:true, beanBg:true` .
- **Rendering without focus.** A hidden (`setVisible(false)`) or covered view stops rendering (IntersectionObserver never fires: the mock's comments never fill, Bean ends `not-found`); so a `beanBg` tab is VISIBLE and top-most at its normal size but at `x = width-1` (a 1 px column shows, the rest is clipped), `backgroundThrottling:false` . No CDP debugger is attached (Google sign-in rejects debugged browsers). Activating it by hand makes it a normal tab again.
- **Tests.** R63: a whole run (2 comments x 2 channels incl. a switch) with the window blurred: 0 focus/raise calls (webContents.focus, window focus/show/moveTop/restore, app.focus), the active tab never changes, the Bean tab is never focused but is `visible` , all 4 likes land once (about the same time as 1.1.6.3: the control run on 1.1.6.3 shows 2 focus calls, the tab switched and `hasFocus` true). Run with `NOTHROTTLE=1` (no anti-throttling switches). unit11: no hasFocus/visibilityState/document.hidden/focus() in the code.
- **Update/reload survival.** `recoverAfterReload` (popup.js): on page start and, for a paused run, when the record has been silent for 25 s: write `p9bn.ping` ; the tab running the batch answers in `p9bn.alive` (one writer); no answer in 4.5 s (and no navigation grace) = orphan: groups `opening|running` -> `queued` (`recycle+1`), `running` items -> `queued` (`reloaded+1` , log line), `pauseReq` + `pause.where = 'Bean was updated|reloaded'` (the version is kept in `p9bn.ver`), the pause key is written, every registered tab is closed (verified). Resume is the normal Resume. R65 (paused) and R66 (working) reload the extension with `reloadUnpacked` , resume at once and check 4 likes, once each.
- **1.1.7 queue.** The queue (`p9bn.orders`) is plain storage, the dispatcher is rebuilt by whichever page is the executor, and a run that was open at the reload is paused by `recoverAfterReload` (also after a popup/runner adoption); Resume re-opens a fresh tab and the queue goes on (R67).

22. 1.1.7.1 - API job reporting, marker, speed, switch watchdog, messaging that cannot hang, pause

What changed since 1.1.7 (tests in parentheses):

- **API job link survives a reload / restart** (ext17.js `apiGuard` , `reconcile`): results not yet acked are re-sent, then the finish; a local order that is done while the server still says `running` gets its finish; a server job running for this install with no local run is reported `interrupted` (never re-clicked). Reporting has its own 1 s timer, separate from the dispatcher tick (e2e11 4b, unit17).
- **Sent-click marker survives the post-switch verify** (content.js synthetic marker from `sent` / `mk`): a like that was sent is "liked", never "already", no extra reload (R76).
- **Speed**: the 3 s VPN settle only after an observed drop (ext11 `vpnUp`); the owner page publishes `p9bn.vpnok` once a second and the tab's gate / pre-click check read it (no request, no wait); VPN requests are re-sent only after 4 s; the page's own account-menu switch waits 6 s (not 15 s) for the navigation. The fresh-tab reopen after a switch is unchanged.
- **Switch watchdog**: a switch address that never comes back is noticed after ~28 s (45 s via the menu), the container is reopened in a fresh tab, that attempt is not counted as failed, the next one uses the other method (R77).
- **Messaging cannot hang a step** (the root cause section below): live sample first, then 3 bounded requests (4 s each); every awaited step before the click is bounded (`bound` : lock, run record, marker read, identity re-check = `step-timeout`); `vpn-unanswered` / `step-timeout` hand the container back (reopen, same location, nothing clicked); the owner always writes a final answer; progress stamps (`prog` / `step`) in the heartbeat and a runner watchdog that recycles a beating tab with no progress for 45 s (`lib/orders.js` `SILENT_MS`) (e2e11 1.1.7.1 (1)-(4), unit17).
- **Pause**: one steady Pausing -> Paused state (the popup latches what it showed, `pauseUi`); Pause never waits for a stuck item (`pauseStuckStep` : no like sent, 15 s without progress, 10 s after the request -> item back to the queue, tab closed, paused) (R82, unit17).
- Background tabs stay default OFF. R81 hides and throttles the background tab on purpose (test-only `P9_BG_HIDDEN` in the test Panel 9) and checks the run degrades safely.

Root cause of the tab <-> Bean window hangs (plain words)

Bean's tab and Bean's window never talk through runtime messages. The tab writes a request into its container's `chrome.storage` ; Panel 9 copies that into the window's storage (a debounced whole-storage dump, merged per key), the window answers the same way back. That copy is best effort and takes 2-8 s in real runs, and a copy can be late or lost, so the "first send" often looked unanswered (this also happens with background tabs OFF). The step that followed the answer then waited with no limit on one more storage / identity step while the tab's separate heartbeat timer kept saying "alive", so the runner only noticed after its 4-minute item stall. Hidden / throttled tabs make it worse: a really hidden page gets its timers once per second (measured: 50 ms timers took ~0.84 s each), once a minute after 5 minutes, and YouTube's own page code starves too. What Bean does now: no request/answer on the happy path (one-way published sample), bounded retries, bounded steps, a progress-based watchdog, and a fail-safe that never clicks.

22b. 1.1.7.1 - PREFER ACTIVE CHANNEL (approved by Oluchi 12:56 WAT Nov 3)

- One record per container: `p9bn.act.<containerId>` = `{ id, name, at, src }` (the last channel Bean saw ACTIVE there, keyed by the stable channel id; `src` = identity | menu). Written by the tab (`noteActive` in content.js) when the multi-signal pre-click check confirms the channel, when a like / "already" is reached (always after the identity check), when the page session confirms the channel (list read skipped), when the account menu is read (also corrects a wrong record), and after a switch-back/restore.
- `L.activeChannel(listRecord, storedRecord)` (lib/links.js): the NEWER of the channel-list read and the stored record wins; an id that is not in the list is ignored; the answer is ONE id or ''. popup.js `activeOf(cid)` feeds `targets()` (so the popup and the API/spare fill share it, and the active channel is also processed first in its container).
- `pickChannels` (lib/orders.js) is now strict: every container's active channel (at most one per container; a container without a channel list counts as active) is taken BEFORE any channel that needs a switch; switch-needing channels are only chosen for the remainder, by the old cost model (a container already visited for its active channel costs only the switches). Used for the popup quantity, API orders and the spare fill after failures.
- Safety unchanged: identity check before every click, no re-click of a sent like, the fresh-tab reopen after any switch, per-link reservation. A wrong stored record only costs a normal switch (e2e R85).
- Tests: unit17 (selection order, one-active-per-container, spare fill, activeChannel newer-wins); e2e R84 (second run, all four channels ticked, quantity 2, stale channel list: 0 switches vs 2 on 1.1.7) and R85 (wrong stored record: normal switch path, record corrected).

22c. 1.1.7.1 - second root cause: un-timed storage calls in the tab (real runs q8ojt7phmw0iecjy, zo8n4jxzmw0n9e3l, fqv3fi2ymw0ost2a)

Plain words: the tab talks to Panel 9 through chrome.storage, and Panel 9 sits in the middle. A reply can be lost, so an awaited read or write simply never finishes. The tab wrote everything through ONE queue (`patch`), so one lost reply froze every later write: no heartbeats, no results. The one thing that still worked (answering the runner's "alive?" ping) is a plain write that is not in that queue, which is why the runner saw "heartbeat missed" while the page still answered pings, and why a ping answer was wrongly taken as proof of life.

- Every storage call of the tab is bounded and repeated (`stor` : 7 s, 3 tries; reads and whole-key writes are safe to repeat); a call that never gets an answer fails as a step-timeout instead of hanging. Each queued run-record write is bounded (45 s) so one stuck write cannot block the queue.
- In-tab DEADMAN (80 s without progress on an item the page is working on): the page hands the item back itself - nothing clicked: fresh tab, same VPN location; a like SENT: a fresh tab only VERIFIES it. A handed-back page never clicks (alive() is re-checked right before the marker and the click).
- Runner: a tab that answers a ping but has made no progress for 45 s (or waits on a switch page that never came) gets no extra grace; a silent tab after a SENT like is requeued for a verify-only fresh tab (up to twice) before it can end as liked-unverified.
- VPN: a lost FINAL answer (the owner started the step, the answer never arrived) is never "vpn-failed": the owner writes its final answer again after 3, 8, 15 s; the tab accepts the owner's live sample (connected after the request) as done; after 120 s it is `vpn-unanswered` (plain words, container reopened, location kept).
- Click line of an earlier page of the item counts as "a like was sent" (`loggedClick`) even if `sent` and the marker never arrived: the result is liked, never "already".
- `not-registered` : after the click the page showed the like, but a FRESH verify page shows the comment NOT liked: the like did not register on YouTube. Not counted as liked, the marker stays (never re-clicked from that channel), the order's spare-channel fill replaces it. The like count seen before each click is stored (`cnt0`); a later item of the same comment that sees no higher count than an earlier "liked" one logs a warning (`cwarn`), information only.
- Automatic "Pause by itself" (13:28:50 in zo8n4jxzmw0n9e3l): the cause is `recoverAfterReload` : when the page that drives the run is judged gone (the runner adopts a popup run, or a paused tab is silent) it pinged the tab and waited only 4.5 s; Panel 9's mirror needs 2-8 s, so a live tab looked orphaned and the run was set to "paused - Bean was reloaded". Now a tab with a fresh heartbeat or fresh report (<20 s) is never treated as orphaned, and the ping waits 12 s. Not confirmed on the real build (only the log line "paused - Bean was reloaded" would prove it).
- The ping answer no longer counts as progress for a tab waiting for a channel-switch page (R77 passed alone but failed after R76 because the ping "saved" the group once). Tests: e11 (a) lost VPN final answer, (f) lost storage reply once / three times, (e) silent verify page after the click; e2e R86 (not-registered), R84/R85 (prefer active channel); unit17.

23. Lessons (from /workspace/BEAN-LESSONS.md)

Oluchi's hard rules

- Publish/deploy only on Oluchi's explicit approval in her own words. Server (Bean API) changes need approval too.
- Never edit /workspace/appstore/server files before approval (a shared publish sync can push them live). Work on a copy.
- Never pull focus; Bean works in the background.
- Language-independent: no text/label matching on YouTube UI.
- Pace is ONE saved setting for all runs (popup, API, YT Panel).
- The fresh-tab reopen after a channel switch MUST stay: it is what loads the new channel's own like state (otherwise the old channel's state shows). Speed up only around it.

Fixes that worked (keep them)

- 1.1.6.x: multi-signal identity check (session channel id + account menu + header picture) before every click and every "already" mark. This has prevented every wrong like.
- 1.1.6.2: after a switch, continue in a fresh tab; same item keeps its VPN location (no re-rotation).
- 1.1.6.4: runs survive an extension update/reload.
- A sent like is only verified afterwards, never re-clicked (sent-click marker must survive every path, incl. post-switch verify — the 1.1.7 "already" mislabel bug).
- 1.1.7: test waits for long e2e cases must be generous (bypass needed ~263 s; 420 s cap was flaky).

Known pitfalls

- Tab<->runner messages get lost (first send often unanswered; worse with background tabs). Every awaited step needs a bounded timeout (~30 s) and a fail-safe (no click, fresh tab, same location).

- If Bean stops reporting to the server, the job stays "running" (polls renew the lease) and blocks the whole queue.
- Popup status text can go stale (e.g. "switching VPN location" while nothing rotates).
- Pause waits for the current item; a stuck item made pause useless.
- Accounts needing reauth show "fetch failed" reading the channel list.
- Mock benchmarks don't reflect real runs; measure speed on Oluchi's real logs.

Added in 1.1.7.1 (Nov 3 2026)

- The tab <-> Bean window "messages" are NOT runtime messages: they are chrome.storage keys (p9bn.vpnreq / p9bn.vpn) that Panel 9 copies between container sessions (debounced, whole-storage dump, merged per key). Delivery is best effort and takes 2-8 s in real runs; a request/answer needs two hops. Do not put a request/answer on the happy path: the owner page PUBLISHES state one-way (p9bn.vpnok, once a second, with locOk) and the tab only reads it. Requests are the fallback.
- A heartbeat is not progress. The tab's heartbeat is its own timer and keeps beating while the main flow waits for an answer that never comes. Carry a progress stamp (prog/step: finished sleeps, log lines, status texts) in the heartbeat; the runner recycles a beating tab with no progress for 45 s (nothing clicked). Do not let helper code (storage patches) tick the progress.
- Every await before a click is bounded (lock, run record, marker read, identity re-check: step-timeout). An unanswered / timed-out step is NEVER a result: no click, the container is handed back (fresh tab, same location). After 2 reopens it fails with a plain message.
- The owner page must always write a final answer for a request (a throw used to leave the tab on "working" for ever).
- Status text must name what is really happening (check / connect / switch); never "switching the VPN location" for a read.
- Pause UI: the popup latches what it showed (Pausing -> Paused; Resume = running for 6 s) because older records come back from the mirror; Pause never waits for a stuck item (cancelled after 10 s + 15 s of no progress, no click, tab closed).
- Hidden, throttled tabs (document.hidden with background throttling): measured 50 ms timers taking ~0.84 s each, rAF never fires; Chromium goes to once a minute after 5 minutes hidden. Everything in the tab is timer polling, so a hidden throttled tab is 10-100x slower. Keep bgTabs default OFF; test with P9_BG_HIDDEN (test-only).
- Test video ids must be exactly 11 characters (HANGPRE_001, not HANGPRE_0001).
- Duplicate sends are not free: every storage write is mirrored to every container; resend a lost request only after ~4 s.
- (1.1.7.1) CLAIMED-IDLE JOBS (real job_V4Drrn4Psg2ovX5T, 16:04 WAT): the server hands a claimed job out ONCE and renews its lease on every poll, so a hand-over that Bean dropped silently (accept() returned early: not the executor for a moment, the queue lease elsewhere, a run still tearing down) left the job "running" with no results for 18 min and blocked the queue. Rules: every claimed job is kept (S17.claims) until it is queued locally; retried every 3 s; after 30 s without a local order it is handed back with a finish report (error not_started, nothing clicked); a job the server calls running that Bean knows nothing about (lost claim answer) is handed back after ~30-45 s; polls never imply progress. Server side (proposal only, not applied): renew a lease only from a report, and give a job that was claimed but has no report within ~60 s back to the queue.
- (1.1.7.1) Pause must be fast: after the press, an item with no progress for 10 s is cut after 4 s (nothing clicked again; a like already sent stays marked so a fresh tab only VERIFIES it). Never wait minutes on "the like in progress finishes first" when that step is hung.
- (1.1.7.1) The popup must show ONE active run: Panel 9's storage mirror can bring back the record of an older run (the job that just finished) after the new one started. A record of a different, older run (created earlier) than the one shown is never shown (staleSeen counter).
- (1.1.7.1) The ~5 minutes between a claim and the first like (real: 16:22:09 -> 16:27:28, 17:00:28 -> 17:05:18) matches ONE frozen first step (4 min stall + reopen) on 1.1.7; with the 45 s watchdog + 80 s in-tab deadman the worst case is ~1-1.5 min. Every run log now has a "timing" line (order waited N s in the queue).
- (1.1.7.1) Test harness: reloading the extension (mgr.reloadUnpacked, e11 section 4b) leaves the hidden runner page of the OLD instance alive but dead ("Extension context invalidated"): jobs then sit "running" with no local run. Rebuild the runner (P9_NO_BEAN_RUNNER toggle) before the next API section; this is a harness artefact, not a product bug.

24. 1.1.7.1 - claimed jobs, fast Pause, one active run (late additions, Nov 3 evening)

- ext17.js `S17.claims / retryClaims() / CLAIM_GIVE_UP_MS = 30000` : a job the server claimed for this install is never dropped silently. `accept()` records it, every early return leaves the claim in place, `apiGuard` (own 1 s timer) retries every 3 s and after 30 s reports it finished with error `not_started` (nothing clicked). `reconcile()` (every 15 s) hands back a running server job that no order and no claim knows after 14 s / 2 sightings and 30 s of age.
- popup.js `pauseStuckStep` : 4 s after Pause, every open item silent for 10 s -> items go back to the queue (sent likes keep `sent` + `verify` : verify-only after Resume), tab closed, run Paused at once.
- popup.js `onChanged` : a record of a different, OLDER run (`created` earlier than the shown run) is ignored (`staleSeen`).
- ext17.js `startBatch` : one "timing" event per run: how long the order waited in the queue.

- Tests: e2e11 section 4a (race: first 3 hand-over attempts fail then the retry starts the job; never-startable job handed back <= 50 s; lost claim handed back), e2e-safety R82 (Paused <= 20 s after the press), R88 (older run record never shown), unit17 checks.

1.1.7.5 - three containers at once (waves) and the step table (Nov 6 2026)

Oluchi: "handle in batches of 3 containers simultaneously since it functions with background on; wait for the channels in the 3 containers to finish and then move to the next three" + "make sure each step has no issues".

- `lib/steps.js` (loaded before popup.js): `PARALLEL_CONTAINERS = 4`, `WAVE_MODE = true`, `STAGGER_MS = 2500`, the STEP TABLE and `pickOpen(b, {now, cap, wave, lastOpenAt})`.
- **Waves.** `advance()` no longer waits while ANY group is open. It asks `pickOpen()` which queued group may open: at most `cap` (4) groups `opening` / `running`; a wave is the set of up to 4 containers opened together (`b.wave = {n, cids}`, also kept in the page's memory `waveMem` because a stale mirror copy can lose it). The next wave starts only when no container of the current one is open and none of its members is queued again. A member that is handed back (fresh tab, reopen) is reopened inside its own wave first. Members are opened one per pass, `STAGGER_MS` apart; the pace gap (`nextAt`) applies between waves only. A given-up member (open-failed, google-check, blacklisted `badCid`, ...) is just `done` and frees its slot; it never blocks the others. Safety net: with nothing open, a queued group is always returned (no stall possible; a 40-run random simulation in tests/unit175.js checks it).
- **VPN rule.** ExpressVPN is ONE connection for the whole PC: a rotation for one container moves the exit address of every container. So at most ONE open container uses the VPN (`usesVpn : b.vpn.on` and an unfinished item that is not `vpnSkip`), and a wave holds at most one VPN container; the other slots are filled with bypassed / VPN-off containers. With VPN rotation off for the run (or every container bypassed) all 4 slots are used. With rotation on and every container on the VPN the run is one container at a time as before (every like keeps its own verified location: never a shared or moving location under a click).
- **Per-container tab rule.** "Never two tabs" is now "never two tabs of ONE container": before an open, `closeExcept(b, cid)` + `sweepVerified(keepOthers(b, cid))` close everything of this run except the tabs of the OTHER open containers; a VPN-drop reopen or a lost tab closes only that container's tab (`closeOneGroup`). `P9BEAN_ORD.tabGone(b, now, {cid})` re-queues only that container.
- **Per-container answers.** The tab answers the alive ping also on `p9bn.alive.<container>` (with `cid`), and `tabAnswers(cur, cid)` only believes that container's answer (one live tab must not hide a dead one). The click lock is `p9bn.lock.<container>` (its job is duplicate pages of ONE container; a global key made the parallel tabs wait on each other's mirrored lock until busy-lock).
- **Record-quiet gate.** advance() used to act only on a record unchanged for ~1 s; with 3 tabs reporting it may never be quiet, so after `QUIET_GATE_MAX_MS` (4 s) it acts on the newest record anyway.
- **Only with Background tabs ON** (`p9bn.bgTabs`, Oluchi's mode): with it off the cap is 1 (foreground tabs would take turns in front; Bean never pulls focus).
- **Fallback without a build:** storage key `p9bn.parallel` = 1, 2 or 3 lowers the number of containers at once (1 = the old one-at-a-time behaviour). Default 4.
- **Steps (name / limit / on timeout)** - popup-side ones go through `ST.step()` (rejects `{bound, step}`; the open path hands the container back, max 3, never a result): vpn-rotate 120 s · vpn-up 310 s · close-previous 20 s · verify-closed 25 s (8 passes) · open-tab 30 s · first-heartbeat 45 s (120 s after a rotation) · channel-switch / find-comment / next-item 45 s no progress (runner recycles, same location, nothing clicked) · identity-check 40 s · click-lock 35 s · like-verify 20 s silent / 45 s hard (liked-unverified, never re-clicked) · tab-deadman 80 s · group-done 20 s (channel restore) · record-quiet 4 s.
- Known residual: the one-shot URL hand-over `p9bn.bind` is still one key; two containers navigating to the SAME link within ~2-8 s could adopt the other's hand-over on a token-less page. The multi-signal identity check before every click/already then refuses (wrong channel identity -> no click, fresh tab), so it can cost time, never a wrong like.